YAPAY ZEKÂYA GİRİŞ

8. ÜNİTE
BİRİNCİ DERECEDEN MANTIK

First-Order Logic (FOL)



Türkçe çeviri, ayrıntılı konu anlatımı, mantıksal formüller, görseller, doğruluk tabloları ve çözümlü örnekler

İstanbul 29 Mayıs Üniversitesi | Yapay Zekâya Giriş

Kaynak kapsamı: Kullanıcı tarafından paylaşılan 8.1–8.4.2 alt bölümleri

Ekim 2026

İçindekiler

Bölüm 8. Birinci Dereceden Mantık **3**

8.1 Bilgi Temsilinin Yeniden İncelenmesi (Representation Revisited) **4**
8.1.1 Düşüncenin Dili (The Language of Thought) . . . 5
8.1.2 Biçimsel ve Doğal Dillerin En İyi Özelliklerini Birleştirmek . . . 6
Ontolojik bağlılık ve epistemolojik bağlılık . . . 7

8.2 Birinci Dereceden Mantığın Sözdizimi ve Anlambilimi **8**
8.2.1 Birinci Dereceden Mantığın Modelleri (Models for First-Order Logic) . . . 8
8.2.2 Semboller ve Yorumlamalar (Symbols and Interpretations) . . . 9
Şekil 8.3 — FOL sözdiziminin biçimsel özeti . . . 10
8.2.3 Terimler (Terms) . . . 11
8.2.4 Atomik Cümleler (Atomic Sentences) . . . 11
8.2.5 Karmaşık Cümleler (Complex Sentences) . . . 12
8.2.6 Nicelik Belirleyiciler (Quantifiers) . . . 12
Evrensel niceleme: ∀ . . . 12
Varoluşsal niceleme: ∃ . . . 13
İç içe niceleyiciler (Nested Quantifiers) . . . 13
∀ ile ∃ arasındaki bağlantılar . . . 14
8.2.7 Eşitlik (Equality) . . . 15
8.2.8 Alternatif Bir Anlambilim? (An Alternative Semantics?) . . . 15

8.3 Birinci Dereceden Mantığın Kullanımı (Using First-Order Logic) **18**
8.3.1 Birinci Dereceden Mantıkta Bildirimler ve Sorgular . . . 18
8.3.2 Akrabalık Alanı (The Kinship Domain) . . . 19
8.3.3 Sayılar, Kümeler ve Listeler (Numbers, Sets, and Lists) . . . 20
Doğal sayılar ve Peano aksiyomları . . . 20
Kümeler . . . 21
Listeler . . . 22
8.3.4 Wumpus Dünyası (The Wumpus World) . . . 23

8.4 Birinci Dereceden Mantıkta Bilgi Mühendisliği **26**
8.4.1 Bilgi Mühendisliği Süreci (The Knowledge-Engineering Process) . . . 26
1. Görevi belirleme (Identify the Task) . . . 26
2. İlgili bilgileri toplama (Assemble the Relevant Knowledge) . . . 27
3. Yüklem, fonksiyon ve sabit sözlüğünü seçme (Decide on a Vocabulary) . . . 27
4. Genel alan bilgisini kodlama (Encode General Knowledge) . . . 27
5. Somut problem durumunu kodlama (Encode the Specific Instance) . . . 27
6. Sorgulama ve cevap alma (Pose Queries) . . . 27
7. Bilgi tabanını hata ayıklama (Debug the Knowledge Base) . . . 27
8.4.2 Elektronik Devreler Alanı (The Electronic Circuits Domain) . . . 28
Adım 1 — Görevi belirleme . . . 28

Adım 2 — İlgili bilgileri toplama . . . 29
Adım 3 — Sembol dağarcığını belirleme . . . 29
Adım 4 — Genel alan aksiyomlarını kodlama . . . 29
Mantık kapılarını hatırlama . . . 31
Adım 5 — Şekil 8.6'nın özel durumunu kodlama . . . 32
Adım adım tam toplayıcı örneği . . . 33
Bütün girişlerin doğruluk tablosu . . . 33
Adım 6 — Sorgular sorarak devreyi doğrulama . . . 33
Adım 7 — Bilgi tabanını hata ayıklama . . . 34

Ek A. Bütün Bölümü Birleştiren Çözümlü Örnekler **35**
Örnek 1 — Genel cümleyi mantığa çevirme . . . 35
Örnek 2 — En az bir nesne . . . 35
Örnek 3 — Niceleyici sırası . . . 35
Örnek 4 — Tam olarak iki nesne . . . 36
Örnek 5 — Aksiyom mu teorem mi? . . . 36
Örnek 6 — Esinti ve belirsizlik . . . 36
Örnek 7 — Tam toplayıcıda üç adet 1 . . . 36

Ek B. Hızlı Başvuru Sözlüğü **37**

Ek C. Öğrenciler İçin Alıştırmalar ve Kısa Cevap Anahtarı **38**

Bölüm 8. Birinci Dereceden Mantık

İlk olarak dünyada çok sayıda nesne bulunduğunu, bunların bazılarının başka nesnelerle ilişkili olduğunu fark ediyor; ardından bu nesneler ve ilişkiler hakkında akıl yürütmenin yollarını araştırıyoruz.

Yedinci Bölüm'de bilgi tabanlı bir ajanın çalıştığı dünyayı temsil etmesi ve hangi eylemi yapacağına ilişkin çıkarımda bulunması incelendi. Mantığın temel kavramlarını anlatmak için önermeler mantığı yeterliydi. Buna karşılık önermeler mantığı, çok sayıda nesne içeren karmaşık dünyalar hakkında bilgiyi **kısa, öz ve genel** biçimde ifade etmekte güçsüz kalır.

Bu bölümde **birinci dereceden mantık (First-Order Logic, FOL)** incelenmektedir. FOL, günlük yaşamdan bildiğimiz pek çok olguyu ifade edebilecek kadar güçlüdür; matematik, felsefe ve yapay zekâda önemli bir yere sahiptir. Diğer bilgi temsil dillerinin bir bölümünü kapsar veya onların temelini oluşturur. Önce temsil dilleri üzerinde duracak, daha sonra FOL'nin sözdizimini ve anlamını öğrenecek, son olarak aile ilişkileri, sayılar, kümeler, Wumpus dünyası ve elektronik devrelerde kullanımını göreceğiz.

Öğrenme hedefi. Bölüm sonunda bir modelin evrenini ve yorumlamasını tarif edebilmeli; sabit, yüklem, fonksiyon ve terimleri ayırt edebilmeli; $\forall$ ve $\exists$ kullanarak cümleleri mantığa çevirebilmeli; verilen aksiyomlardan sorgu cevapları çıkarabilmeli ve bir bilgi tabanının nasıl geliştirildiğini açıklayabilmelisiniz.

8.1 Bilgi Temsilinin Yeniden İncelenmesi (Representation Revisited)

Bu alt bölüm temsil dillerinin doğasını ele alır. Amaç, 7. Bölüm'deki önermeler mantığından çok daha güçlü bir dil olan birinci dereceden mantığa neden ihtiyaç duyulduğunu anlamaktır. Bunun için hem önermeler mantığının hem programlama ve doğal dillerin hangi noktalarda işe yaradığını, hangi noktalarda sınırlı kaldığını düşünürüz.

C++, Java ve Lisp gibi **programlama dilleri**, günümüzde yaygın kullanılan biçimsel dillerin en büyük grubunu oluşturur. Bir program doğrudan hesaplama süreçlerini temsil eder; programdaki veri yapıları ise dünyaya ilişkin olguları tutabilir. Wumpus dünyasını 4×4 bir dizide temsil eden bir programda

```
World[2,2] <- Pit
```

satırı, [2,2] konumunda bir çukur bulunduğunu bildirebilir. Bu tür veri yapıları belirli bir iş için tasarlanmıştır. Veritabanları, olguları saklama ve geri alma işini daha genel ve alan bağımsız hale getirmek için geliştirilmiştir.

Buna rağmen sıradan programlama dillerinde, bir olgudan başka olgular türetmeye yarayan **alan bağımsız, genel bir çıkarım mekanizması** dilin kendisinin parçası değildir. Veri yapısı güncellemeleri genellikle programcının alan bilgisini kodladığı özel yordamlarla gerçekleştirilir. Prosedürel yaklaşımın karşısında, önermeler mantığının **bildirimsel (declarative)** niteliği bulunur: Bilgi ile o bilgiden çıkarım yapma yöntemi birbirinden ayrılmıştır.

İkinci sorun **kısmi bilgidir**. Örneğin "Çukur [2,2] veya [3,1] konumundadır" yahut "Wumpus [1,1]'deyse [2,2]'de değildir" cümlelerini ele alalım. Değişkenlere tek değer atayan sıradan veri yapıları, söz konusu belirsizliği mantıksal anlamıyla doğrudan temsil etmekte yetersiz olabilir. Önermeler mantığında VEYA ve DEĞİL bağlaçlarıyla kısmi bilgi ifade edilebilir.

Önermeler mantığının üçüncü yararlı özelliği **bileşimsellik (compositionality)** ilkesidir. Bileşimsel bir dilde karmaşık bir cümlenin anlamı, bileşenlerinin anlamlarından elde edilir. Örneğin $S_{1,4}$ ifadesi [1,4]'te, $S_{1,2}$ ifadesi [1,2]'de kötü koku olduğunu söylüyorsa $S_{1,4} \wedge S_{1,2}$ iki olgunun da geçerli olduğunu anlatır; bambaşka bir spor karşılaşmasının sonucunu anlatamaz. Bileşimsellik olmadan bir akıl yürütme sisteminin işi çok zorlaşır.

Bununla birlikte önermeler mantığı, pek çok nesne içeren dünyaları özlü biçimde açıklama gücünden yoksundur. Wumpus dünyasında her kare için ayrı bir esinti–çukur kuralı yazmak zorunda kalmıştık:

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}).$$

Oysa doğal dilde tek cümle yeterlidir: **"Çukura komşu kareler esintilidir."** Doğal dilin nesneler ve genellemeler üzerinde rahatça konuşma özelliğini biçimsel mantığa kazandırmak isteriz.

Kısa karşılaştırma. Önermeler mantığıyla $StudentAli \Rightarrow ExamAli$ ve $StudentAyse \Rightarrow ExamAyse$ gibi farklı kişiler için ayrı ifadeler gerekir. FOL ile tek bir kural yazılır: $\forall x\ (Student(x) \Rightarrow Exam(x))$. Bu, "Bütün öğrenciler sınava girer" anlamındadır.

8.1.1 Düşüncenin Dili (The Language of Thought)

İngilizce ve İspanyolca gibi doğal diller son derece ifade gücü yüksek dillerdir; hemen hemen bütün bir ders kitabını doğal dille yazmak mümkündür. Dilbilim ve dil felsefesinde doğal dilin bildirimsel bir bilgi temsil aracı olduğunu savunan uzun bir gelenek bulunmaktadır. Doğal dilin kurallarını tam olarak çözebilirsek yazılı milyarlarca sayfalık bilgiyi yapay akıl yürütme sistemlerine kazandırabiliriz.

Bununla birlikte modern bakış açısında doğal dil yalnızca saf bir temsil değil, öncelikle **iletişim aracıdır**. Bir kişinin bir noktayı işaret ederek "Bak!" demesi, dinleyiciye binaların üzerinde beliren Superman'i fark ettirebilir; fakat "Bak!" sözcükleri tek başına "Superman çatıda belirdi" olgusunu kodlamaz. Anlam hem söylenen ifadeye hem **bağlama** bağlıdır. Bağlam saklanmazsa "Bak!" cümlesini bilgi tabanına kaydetmek daha sonra aynı anlamı bulmaya yetmez. Bağlamı nasıl temsil edeceğimiz de başlı başına bir sorudur.

Doğal dilin başka bir sorunu **çok anlamlılık / belirsizlik (ambiguity)** olgusudur. Örneğin İngilizcedeki *spring* kelimesi hem ilkbaharı hem mekanik yayı ifade edebilir. Pinker'ın (1995) vurguladığı gibi insanlar düşündükleri kavramı bu iki anlamla karıştırmazlar; dolayısıyla bir kelime ile zihindeki bir düşünce arasında zorunlu olarak bire bir eşleme bulunmaz.

Sapir–Whorf hipotezi, konuşulan dilin dünyayı anlama biçimimizi güçlü şekilde etkilediğini öne sürer. Whorf'a (1956) göre doğayı kavramlara bölme ve anlamlandırma alışkanlıklarımız dil topluluğumuzun paylaşılan düzenlemelerinden etkilenir. Örneğin Fransızcada İngilizcedeki *chair* sözcüğünün kapsadığı alanda *chaise* ve *fauteuil* gibi ayrımlar bulunur. Ancak İngilizce konuşan bir kişinin kollu koltuğu tanıyıp başka ifadelerle adlandırabilmesi, dilin düşünce üzerindeki etkisinin tam olarak ne düzeyde olduğunu tartışmalı hale getirir.

Sonraki araştırmalar yalnız sezgilere değil antropolojik, psikolojik ve nörolojik veriye de dayanmıştır. **Wanner (1974)**, okunan bir metnin hangi sözcüklerle ifade edildiğini hatırlamaya ilişkin deneylerde katılımcıların iki alternatif arasındaki doğru seçimi yaklaşık şans düzeyinde (%50) yaptığını, metnin içeriğini ise %90'ın üzerinde doğrulukla hatırlayabildiğini göstermiştir. Bu bulgu, kelimelerin işlenmesinden sonra sözel olmayan bir anlam temsilinin oluşabileceğini düşündürür.

Bazı dillerde belirli kavramların bulunmaması da önemlidir. Avustralya'nın yerli dillerinden **Guugu Yimithirr**, ön, arka, sağ ve sol gibi göreli yönler yerine kuzey, güney, doğu ve batı gibi mutlak yönleri kullanır; buna karşılık gelen "Kuzey kolum ağrıyor" türü ifadeler üretilebilir. Kaynakta aktarılan araştırmalar, bu dili konuşanların açık arazide yön bulmada, İngilizce konuşanlarınsa çatalı tabağın sağına koymak gibi göreli konum görevlerinde avantaj gösterebildiğini ifade eder.

Dilbilgisel cinsiyetin etkileri de incelenmiştir. "Köprü" İspanyolcada eril, Almancada dişil isimdir. **Boroditsky (2003)** tarafından aktarılan çalışmada köprüyü İngilizce sıfatlarla tarif eden İspanyolca konuşanlar "büyük, tehlikeli, güçlü, heybetli", Almanca konuşanlar "güzel, zarif, kırılgan, ince" gibi sıfatlara daha sık yönelmiştir. Kelimelerin algıda birer dayanak noktası işlevi görebileceği savunulur.

Loftus ve Palmer (1974), katılımcılara aynı trafik kazası videosunu gösterip çarpışmayı farklı kelimelerle sorduklarında hız tahminlerinin değiştiğini bulmuştur. "Temas etti" (*contacted*) sözcüğüyle sorulanlarda yaklaşık **32 mil/saat**, "şiddetle çarpıştı" (*smashed*) sözcü-

ğüyle sorulanlarda yaklaşık **41 mil/saat** bildirilmiştir. Soru dilindeki değişiklik değerlendirmeyi etkileyebilmiştir.

Birinci dereceden mantıkta, şu iki ifade De Morgan kuralları nedeniyle eşdeğerdir:

$$\neg(A \vee B) \equiv (\neg A \wedge \neg B).$$

CNF (Birleşik Normal Form) kullanan bir akıl yürütme sistemi bu iki ifadeyi aynı standart yapıyla saklayabilir. İnsan beyninde de benzer bir temsile bakabilir miyiz? **Mitchell ve arkadaşları (2008)** kişilere "kereviz" gibi sözcükler gösterirken fMRI görüntüleri kaydedip hangi kelimenin gösterildiğini tahmin eden bir program eğitmiştir. İki seçenek arasında doğruluk yaklaşık **%77** olmuştur. Sistem, eğitimde görmediği kelime ve kişiler için de şans düzeyinin üzerinde tahminler yapabilmiştir. Çalışmalar henüz erken aşamada olsa da fMRI ve beyin içi elektrofizyoloji gibi yöntemler insan bilgi temsilini anlama bakımından umut vaat eder.

Biçimsel mantık açısından aynı bilginin iki eşdeğer biçimde ifade edilmesi **mantıksal olarak türetilebilen sonuçları değiştirmez**. Uygulamada ise bir temsil biçimi sonuca daha az adımda ulaşabilir. Sınırlı hesaplama gücüne sahip bir sistem için temsil seçimi bu yüzden önemlidir. Deneyimden öğrenme gibi tümdengelim dışı süreçlerdeyse sonuçlar doğrudan temsil biçimine bağlıdır: Gözlemle eşit derecede tutarlı teoriler arasından daha kısa olanın seçilmesi, hangi dilde "kısa" dendiğine göre değişir. Bu yüzden öğrenen bir ajan için temsil dilinin etkisinden bütünüyle kaçınmak mümkün değildir.

Ders yorumu. Doğal dilin ifade gücü yüksektir; fakat bağlam ve çok anlamlılık nedeniyle belirsizdir. Biçimsel mantığın belirsizliği azaltılmış, açıkça tanımlanmış bir anlam yapısı vardır; fakat önermeler mantığı özlü genellemeler için zayıftır. FOL bu iki dünyanın güçlü yönlerini bir araya getirmeye çalışır.

8.1.2 Biçimsel ve Doğal Dillerin En İyi Özelliklerini Birleştirmek

Birinci dereceden mantık tasarlanırken önermeler mantığının **bildirimsel, bileşimsel, bağlamdan bağımsız ve açık anlambilimi** temel alınır; doğal dilin nesneler ve ilişkiler hakkında konuşabilme yeteneği de bu temele eklenir. İsimler ve isim öbekleri nesnelere (kare, çukur, Wumpus), fiiller ve fiil öbekleri ilişkilere (komşu olma, ateş etme, sahip olma) karşılık gelir. Bazı ilişkiler, her girdinin tek bir değere karşılık geldiği **fonksiyonlar** olarak ele alınabilir.

- **Nesneler:** İnsanlar, evler, sayılar, teoriler, Ronald McDonald, renkler, beyzbol maçları, savaşlar, yüzyıllar.
- **İlişkiler ve özellikler:** Kırmızı, yuvarlak, sahte, asal, çok katlı gibi tekli özellikler; kardeşi olmak, daha büyük olmak, içinde bulunmak, parçası olmak, bir renge sahip olmak, sonra gerçekleşmek, sahip olmak, arada bulunmak gibi çok nesneli ilişkiler.
- **Fonksiyonlar:** Birinin babası, en iyi arkadaşı, bir maçın üçüncü devresi, bir sayının bir fazlası, bir olayın başlangıcı.

Neredeyse her olgu nesnelere, özelliklere ve ilişkilere gönderme yapılarak ifade edilebilir:

1. **"Bir artı iki, üçe eşittir."** Nesneler: bir, iki, üç, "bir artı iki"; ilişki: eşitlik; fonksiyon: toplama. $Plus(1, 2)$ bir nesneyi ifade eder, 3 de aynı nesnenin adıdır.
2. **"Wumpus'a komşu kareler kokuludur."** Nesneler: Wumpus ve kareler; özellik: kokulu olmak; ilişki: komşuluk.
3. **"Kötü Kral John, 1200 yılında İngiltere'yi yönetti."** Nesneler: John, İngiltere ve 1200; ilişki: yönetmek; özellikler: kötü ve kral olmak.

Birinci dereceden mantık, evrendeki **bazı** veya **bütün** nesneler hakkında ifadeler de kurabilir. “Wumpus’a komşu her kare kokuludur” gibi bir genel yasa yazabilmenin önemi buradadır.

Ontolojik bağlılık ve epistemolojik bağlılık

Bir dilin **ontolojik bağlılığı (ontological commitment)**, dünyanın yapısı hakkında neyin var olduğunu varsaydığıdır. Önermeler mantığı doğru/yanlış olguları varsayar; FOL olgulara ek olarak nesneleri ve nesnelerin ilişkilerini varsayar. **Zamansal mantık**, olguların geçerli olduğu sıralı zamanları da içeri alır. **Yüksek dereceli mantık** ise ilişkileri ve fonksiyonları da nesneler gibi ele alarak ilişkilerin kendileri üzerinde genelleme yapılmasına izin verir; bazı ifadeleri sonlu sayıdaki FOL cümlesiyle dile getirmek mümkün değildir.

Epistemolojik bağlılık (epistemological commitment), bir ajanının bir olguya ilişkin hangi bilgi durumlarına sahip olabileceğidir. Önermeler mantığı ve FOL’de bir cümlenin doğru olduğu, yanlış olduğu veya hangisi olduğunun bilinmediği durumları ayırt ederiz. Bu “bilinmiyor” durumu yeni bir doğruluk değeri değil, ajanın bilgisel konumudur. **Olasılık teorisi** $[0, 1]$ arasında *inanç derecesi* kullanabilir (örneğin Wumpus’un [1,3]’te olma olasılığı $0{,}75$). **Bulanık mantık** ise *doğruluğun derecesini* ifade eder (“Viyana büyük bir şehirdir” olgusunun doğruluk derecesi örneğin $0{,}6$ olabilir). İnanç derecesi ile doğruluk derecesi karıştırılmamalıdır.

Şekil 8.1 — Dillerin ontolojik ve epistemolojik bağlılıkları

Dil	Dünyada varsayılanlar	Bilgiye ilişkin durum
Önermeler mantığı	Olgular	Doğru / yanlış / bilinmiyor
Birinci dereceden mantık	Olgular, nesneler, ilişkiler	Doğru / yanlış / bilinmiyor
Zamansal mantık	Olgular, nesneler, ilişkiler, zamanlar	Doğru / yanlış / bilinmiyor
Olasılık teorisi	Olgular	İnanç derecesi: $[0, 1]$
Bulanık mantık	Dereceli doğruluğu olan olgular	Doğruluk derecesi (bilinen aralık değeri)

FOL öğrenirken onlarca gösterim biçiminin ayrıntılarına saplanmamak gerekir. Önemli olan **özlü temsil** ile **mantıksal olarak güvenilir çıkarım** arasındaki ilişkidir. Yapay zekâ öğrencisi mantıksal gösterimleri rahatça okuyup yazabilmelidir.

8.2 Birinci Dereceden Mantığın Sözdizimi ve Anlambilimi

Bu bölüm FOL dünyalarının neden **nesneler ve ilişkiler** içerdiğini biçimsel hale getirir; ardından dilin sözdizimini (syntax), yani hangi dizilerin anlamlı cümleler olduğunu, ve anlambilimini (semantics), yani bu cümlelerin hangi koşullarda doğru olduğunu açıklar.

8.2.1 Birinci Dereceden Mantığın Modelleri (Models for First-Order Logic)

7. Bölüm'den bir mantık dilinin **modellerinin**, olası dünyaların biçimsel gösterimleri olduğunu hatırlıyoruz. Her modelde dilin sembollerini dünyadaki unsurlarla ilişkilendiren bir yapı bulunur; bu sayede bir cümlenin doğru mu yanlış mı olduğunu kontrol ederiz. Önermeler mantığında semboller doğrudan doğruluk değerlerine bağlanırken FOL modelleri **nesneler** içerir.

Bir modelin **evreni (domain)**, içerdiği nesneler, yani *evren elemanları* kümesidir. Evren **boş olamaz**: Her olası dünyada en az bir nesne yer alır. Matematiksel olarak nesnelerin gündelik isimlerinden çok sayılarına ve ilişkilerine bakılır; anlatımı kolaylaştırmak için aşağıdaki somut örneği kullanacağız.

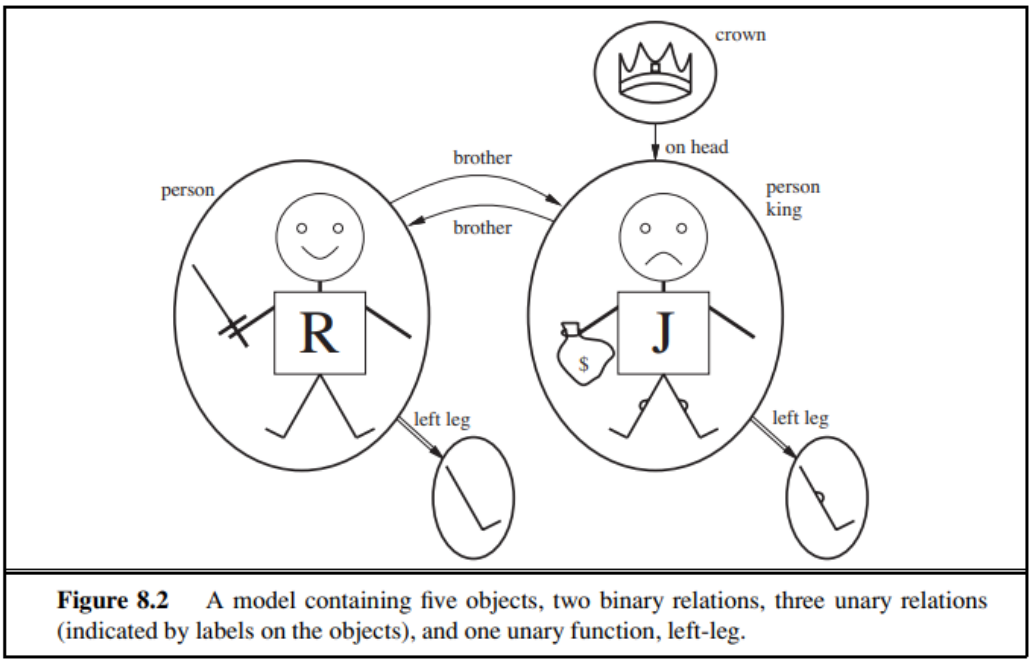


Figure 8.2 A model containing five objects, two binary relations, three unary relations (indicated by labels on the objects), and one unary function, left-leg.

Şekil 8.2. Beş nesneli örnek model; iki ikili ilişki, üç tekli özellik ve bir LeftLeg fonksiyonu bulunmaktadır. Orijinal şekil kullanıcı tarafından sağlanmıştır.

Şekil 8.2'deki beş nesne: 1189–1199 yıllarında hüküm sürmüş Aslan Yürekli Richard, onun kardeşi ve 1199–1215 yıllarında hüküm sürmüş Kral John, Richard'ın sol bacağı, John'un sol bacağı ve bir taç. Şekilde R ve J harfleri iki kişiyi gösterir.

İlişki, matematiksel anlamda bir arada ilişkili bulunan **sıralı nesne demetlerinin (tuples)** kümesidir. Demet, sırası belirlenmiş nesneler listesi olup $\langle\ \rangle$ ile yazılır. Kardeşlik ilişkisi:

$$Brother = \{\langle Richard, John\rangle, \langle John, Richard\rangle\}.$$

Taç yalnız John'un başında olduğu için:

$$OnHead = \{\langle CrownObject, John\rangle\}.$$

`Brother` ve `OnHead` **ikili ilişkilerdir**; her demette iki nesne vardır. **Tekli ilişkiler / özellikler** ise bir nesne üzerinde geçerlidir: `Person` hem Richard hem John için, `King` yalnız John için, `Crown` yalnız tacın kendisi için doğrudur. Modelin anlık durumunda Richard artık hayatta olmadığından kral sıfatının yalnız John'a ait olması amaçlanmıştır.

Bazı ilişkileri **fonksiyon** olarak ele almak uygundur: Modelde her kişiye **tam olarak bir** sol bacak nesnesi eşleyen $LeftLeg$ fonksiyonu tanımlanır:

$$LeftLeg(Richard) = RichardsLeftLeg, \qquad LeftLeg(John) = JohnsLeftLeg.$$

Birinci dereceden mantığın standart fonksiyonları teknik olarak **toplam (total)** olmalıdır: Tanım kümesinin her girdisinde bir çıktı değerine sahip olmalıdır. Bu durumda tacın ve iki bacak nesnesinin de `LeftLeg` sonucunun bulunması gerekir; gündelik dünyada bu tuhaftır. Kitap, sol bacağı olmayan her nesnenin sol bacağı olarak sayılabilen, kendi sol bacağı yine kendisi olan ilave "görünmez" bir nesneden söz eder. Bu gibi girdiler üzerine hiçbir olgu yazılmadığında teknik ayrıntı pratik çıkarımı etkilemez.

Modeli küme olarak okuyalım. $D = \{R, J, L_R, L_J, C\}$ olsun. O zaman $Person = \{R, J\}, King = \{J\}, Crown = \{C\}, Brother = \{(R, J), (J, R)\}$ ve $OnHead = \{(C, J)\}$ olur. Fonksiyonun ilgili eşlemeleri $LeftLeg(R) = L_R$ ve $LeftLeg(J) = L_J$ şeklindedir. Burada C bir nesnenin kısa adı; 'Crown(x)' ise bir özelliği temsil eden yüklemdir.

8.2.2 Semboller ve Yorumlamalar (Symbols and Interpretations)

FOL'nin temel sözdizimsel sembolleri üç türdedir:

- **Sabit sembolleri (constant symbols):** Nesneleri temsil eder: `Richard`, `John`.
- **Yüklem sembolleri (predicate symbols):** İlişki veya özellikleri temsil eder: `Brother`, `OnHead`, `Person`, `King`, `Crown`.
- **Fonksiyon sembolleri (function symbols):** Fonksiyonları temsil eder: `LeftLeg`.

Kitap, bu sembolleri büyük harfle başlatma geleneğini benimser. Sembolün ismi keyfîdir; önemli olan ona verilen **yorumlamadır**. Yüklem ve fonksiyonun argüman sayısına **arity** denir. Örneğin `Person(x)` tekli, `Brother(x,y)` ikilidir; `LeftLeg(x)` tek argümanlı fonksiyondur.

Her modelde bir **yorumlama (interpretation)** bulunur. Yorumlama, sabitlerin hangi nesneye, yüklemlerin hangi ilişkiler kümesine, fonksiyonların hangi eşlemelere karşılık geldiğini belirler. Şekil 8.2 için **amaçlanan yorumlamada (intended interpretation)** `Richard`

Aslan Yürekli Richard'a, `John` Kral John'a, `Brother` yukarıdaki kardeşlik demetlerine, `OnHead` taç–John ilişkisine, `LeftLeg` ise sol bacak eşlemesine karşılık gelir.

Ancak farklı yorumlamalar da mümkündür. Örneğin `Richard` sembolü tacı, `John` sembolü John'un sol bacağını gösterebilir. Beş nesne ve iki sabit sembolü olduğundan yalnızca sabit yorumları için $5 \times 5 = 25$ farklı seçim vardır. Ayrıca bütün nesnelerin sabit bir isme sahip olması zorunlu değildir ve **iki ayrı isim aynı nesneyi gösterebilir**. Örneğin hem `Richard` hem `John` sembolünün tacı gösterdiği bir yorumlama geçerlidir. Tutarsız yorumlamaları elemek bilgi tabanındaki aksiyomların görevidir.

Şekil 8.3 — FOL sözdiziminin biçimsel özeti

Aşağıdaki biçim, kitapta verilen Backus–Naur gösteriminin Türkçe açıklamalı karşılığıdır:

```
Sentence        -> AtomicSentence | ComplexSentence
AtomicSentence  -> Predicate | Predicate(Term,...) | Term = Term
ComplexSentence -> (Sentence) | [Sentence]
                   | NOT Sentence
                   | Sentence AND Sentence
                   | Sentence OR Sentence
                   | Sentence IMPLIES Sentence
                   | Sentence IFF Sentence
                   | Quantifier Variable,... Sentence
Term            -> Function(Term,...) | Constant | Variable
Quantifier      -> ALL | EXISTS
Constant        -> A | X1 | John | ...
Variable        -> a | x | s | ...
Predicate       -> True | False | After | Loves | Raining | ...
Function        -> Mother | LeftLeg | ...
```

Mantıksal karşılıklar: NOT $= \neg$, AND $= \wedge$, OR $= \vee$, IMPLIES $= \Rightarrow$, IFF $= \Leftrightarrow$, ALL $= \forall$, EXISTS $= \exists$. İşlem önceliği yüksekten düşüğe $\neg$, $=$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$ şeklindedir. Niceleyici, kendisinden sonra gelen uygun cümlenin tamamına uygulanır; belirsizliği önlemek için parantez kullanmak yararlıdır.

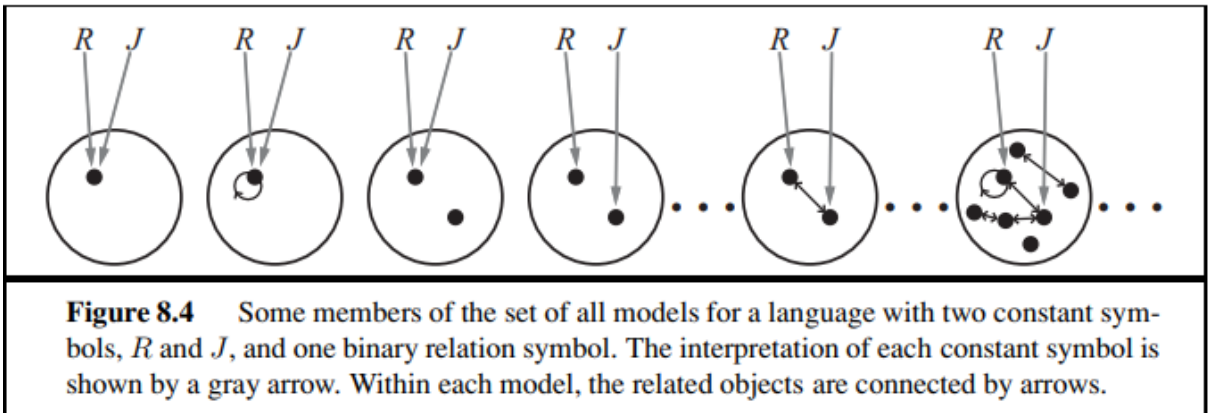


Figure 8.4 Some members of the set of all models for a language with two constant symbols, R and J, and one binary relation symbol. The interpretation of each constant symbol is shown by a gray arrow. Within each model, the related objects are connected by arrows.

Şekil 8.4. İki sabit R,J ve bir ikili ilişki içeren bir dilin olası modellerinden bazıları. Gri oklar sabitlerin yorumunu; nesneler arasındaki oklar ikili ilişkiyi gösterir.

Şekil 8.4, olası modellerin nesne sayılarında, sabitlerin hangi nesneleri gösterdiğinde ve nesnelerin ilişkili olup olmamasında farklılaşabildiğini anlatır. **Tek nesne** olduğunda hem R hem J o nesneyi göstermelidir. Daha fazla nesne olduğunda da R ve J aynı nesneyi gösterebilir; bazı nesneler tamamen isimsiz kalabilir. Bir model bir nesneden sonsuz sayıda nesneye kadar uzanabilir.

Özetle FOL modeli **(i)** nesneler kümesi ve **(ii)** sabitleri nesnelere, yüklemleri ilişkilere, fonksiyonları fonksiyonlara eşleyen yorumlamadan oluşur. Mantıksal sonuç, geçerlilik ve benzeri kavramlar **bütün olası modeller** üzerinden tanımlanır. Modellerin sayısı sınırsız olduğundan önermeler mantığında yapılabilen "bütün modelleri say ve kontrol et" yöntemi genel FOL için pratik değildir. Kitap, Şekil 8.4'teki iki sabit ve tek ikili ilişki dili için **altı veya daha az nesneli 137.506.194.466 model** bulunduğunu belirtir.

8.2.3 Terimler (Terms)

Terim, bir nesneye gönderme yapan mantıksal ifadedir. Sabit sembolleri terimdir; fakat her nesneye ayrı bir sabit vermek gerekmez. Günlük dilde "Kral John'un sol bacağı" deriz; bu nesneyi FOL'de $LeftLeg(John)$ terimiyle adlandırabiliriz. Genel olarak karmaşık bir terim şu biçimdedir:

$$f(t_1, t_2, \ldots, t_n).$$

Burada f fonksiyon sembolüdür, $t_1, \ldots, t_n$ terimlerdir. Terimin anlamı bir **nesnedir**, doğru veya yanlış değeri değildir. Karmaşık terim bir programlama alt yordamının çalıştırılması değil, daha ayrıntılı bir **isimdir**. `LeftLeg` adında çalışan gerçek bir alt program tanımlamadan da sol bacaklarla ilgili mantıksal ifadeler yazabiliriz.

Biçimsel anlamda f sembolü modeldeki F fonksiyonuna, t_i terimleri d_i nesnelerine karşılık geliyorsa bütün terim $F(d_1, \ldots, d_n)$ sonucunda belirlenen nesneye karşılık gelir. `John` Kral John'u gösteriyorsa $LeftLeg(John)$ onun sol bacağını gösterir. Böylece yorumlama her terimin gösterdiği nesneyi sabitler.

> **Terim mi cümle mi?** $Father(Ali)$ bir **terimdir**: "Ali'nin babası" nesnesini gösterir. $Doctor(Father(Ali))$ bir **atomik cümledir**: "Ali'nin babası doktordur" der ve doğru/yanlış olabilir. $Plus(2, 3)$ nesneyi belirleyen terim; $Plus(2, 3) = 5$ ise cümledir.

Kitap, λ ifadeleriyle anında fonksiyon tanımlanabildiğine de değinir: $(\lambda x \ x \times x)$ karesini alma fonksiyonunu gösterir. λ ile yapılan bu tür gösterimler, uygun yerine koymayla ortadan kaldırılabildiklerinden standart FOL'nin biçimsel ifade gücünü artırmaz; bir gösterim kolaylığı sağlar.

8.2.4 Atomik Cümleler (Atomic Sentences)

Terimler ve yüklemler birleştirilerek temel olguları anlatan **atomik cümleler (atoms)** oluşturulur. Örneğin:

$$Brother(Richard, John)$$

"Richard, John'un kardeşidir" anlamındadır. Yüklemin argümanları karmaşık terimler de olabilir:

$$Married(Father(Richard), Mother(John)).$$

Bu ifade, uygun yorumlamada Richard'ın babasının John'un annesiyle evli olduğunu söyler. Bir atomik cümle, yüklemin gösterdiği ilişkinin argüman terimlerinin gösterdiği nesneler arasında geçerli olması halinde doğrudur.

Argüman sırası önemlidir. $Loves(Ali, Ayse)$ ifadesi "Ali, Ayşe'yi sever"; $Loves(Ayse, Ali)$ ifadesi "Ayşe, Ali'yi sever" anlamına gelir. İkinci cümle birinciden otomatik olarak çıkmaz.

8.2.5 Karmaşık Cümleler (Complex Sentences)

Mantıksal bağlaçlar, önermeler mantığındaki sözdizimi ve anlambilimle aynen kullanılabilir. Şekil 8.2'nin amaçlanan yorumlamasında aşağıdakiler doğrudur:

$$\neg Brother(LeftLeg(Richard), John)$$

Richard'ın sol bacağı, John'un kardeşi değildir.

$$Brother(Richard, John) \land Brother(John, Richard)$$

Richard ve John karşılıklı kardeştir.

$$King(Richard) \lor King(John)$$

Richard veya John kraldır (standart VEYA kapsayıcıdır; ikisinin de kral olduğu modellerde de doğru olur).

$$\neg King(Richard) \Rightarrow King(John)$$

Richard kral değilse John kraldır.

Bağlaç	Türkçe okunuş	Kısaca doğruluk koşulu
$\neg P$	P değil	P yanlışsa doğru
$P \land Q$	P ve Q	İkisi de doğruysa doğru
$P \lor Q$	P veya Q	En az biri doğruysa doğru
$P \Rightarrow Q$	P ise Q	Yalnızca P doğru, Q yanlışsa yanlış
$P \Leftrightarrow Q$	P ancak ve ancak Q	Doğruluk değerleri aynıysa doğru

8.2.6 Nicelik Belirleyiciler (Quantifiers)

Bir dil nesneleri temsil edebiliyorsa, nesneleri tek tek isimlendirmeden hepsi veya bazıları hakkında konuşmak isteriz. FOL'de bunun için iki standart niceleyici vardır: **evrensel** $\forall$ ve **varoluşsal** $\exists$.

Evrensel niceleme: $\forall$

7. Bölüm'de "Çukura komşu karelerde esinti vardır" gibi genel kuralları önermeler mantığında yazmak zordu. FOL'de "Bütün krallar insandır" cümlesi tek ifadeye dönüşür:

$$\boxed{\forall x \ (King(x) \Rightarrow Person(x)).}$$

$\forall$ "her" anlamına gelir: "Her x için, x kralsa x insandır." x bir **değişkendir**; küçük harfle yazılması gelenekseldir. Değişken de bir terim olduğu için $LeftLeg(x)$ gibi fonksiyon argümanı olabilir. Değişken içermeyen terime **ground term** denir; $LeftLeg(John)$ örnektir.

$\forall x\ P(x)$, x evrendeki **her** nesneye atandığında $P(x)$ ifadesinin doğru olmasını ister. Teknik tanımda bir değişkene evrenin bir elemanını atayan **genişletilmiş yorumlamalar (extended interpretations)** dikkate alınır.

Şekil 8.2'de beş nesne bulunduğundan $\forall x\ (King(x) \Rightarrow Person(x))$ kontrolü beş kez yapılır: x Richard, John, Richard'ın bacağı, John'un bacağı veya taç olabilir. Yalnızca John kraldır ve aynı zamanda insandır; diğerlerinde $King(x)$ yanlıştır. **Koşullu önermede öncül yanlış olduğunda önerme doğrudur**; bunun için tacın veya bacakların insan olduğunu ispatlamak gerekmez.

x	$King(x)$	$Person(x)$	$King(x) \Rightarrow Person(x)$
Richard	Yanlış	Doğru	Doğru
John	Doğru	Doğru	Doğru
Richard'ın sol bacağı	Yanlış	Yanlış	Doğru
John'un sol bacağı	Yanlış	Yanlış	Doğru
Taç	Yanlış	Yanlış	Doğru

Önemli hata: $\forall x\ (King(x) \wedge Person(x))$ "Bütün krallar insandır" değildir. Bu ifade **evrendeki her nesnenin hem kral hem insan** olmasını zorunlu kılar. "Bütün A'lar B'dir" kalıbında temel bağlaç çoğu kez $\Rightarrow$ olur.

Varoluşsal niceleme: $\exists$

Bir nesnenin var olduğunu ismini söylemeden belirtmek için $\exists$ kullanılır. "John'un başında bir taç vardır":

$$\boxed{\exists x\ (Crown(x) \wedge OnHead(x, John)).}$$

$\exists x\ P(x)$, evrendeki **en az bir** x için $P(x)$ doğrudur demektir. Örneğimizde taç nesnesi `Crown` özelliğini taşır ve John'un başındadır; cümle doğrudur. John iki taç taksaydı da ifade doğru olurdu; "tam olarak bir" dememektedir.

Önemli hata: $\exists x\ (Crown(x) \Rightarrow OnHead(x, John))$ çoğu durumda aşırı zayıf olur. Richard taç değilse $Crown(Richard)$ yanlış olduğundan koşullu önerme doğrudur; başta taç bulunmasa bile varoluşsal ifade doğrulanabilir. "Bazı A'lar B'dir" için genellikle $\wedge$ gerekir.

> **Ezber yerine anlamı kullanın:** "Bütün öğrenciler çalışır" = $\forall x(Student(x) \Rightarrow Studies(x))$. "Çalışan bir öğrenci vardır" = $\exists x(Student(x) \wedge Studies(x))$. Niceleyiciden sonraki bağlacın neden değiştiğini, doğrudan doğruluk koşulları açıklamaktadır.

İç içe niceleyiciler (Nested Quantifiers)

Birden fazla niceleyici içeren cümleler kurulabilir. "Erkek kardeş olanlar kardeştir":

$$\forall x \forall y\ (Brother(x, y) \Rightarrow Sibling(x, y)).$$

Aynı tür iki niceleyici $\forall x, y$ olarak kısaltılabilir. Kardeşlik ilişkisinin simetrik olduğunu söylemek için:

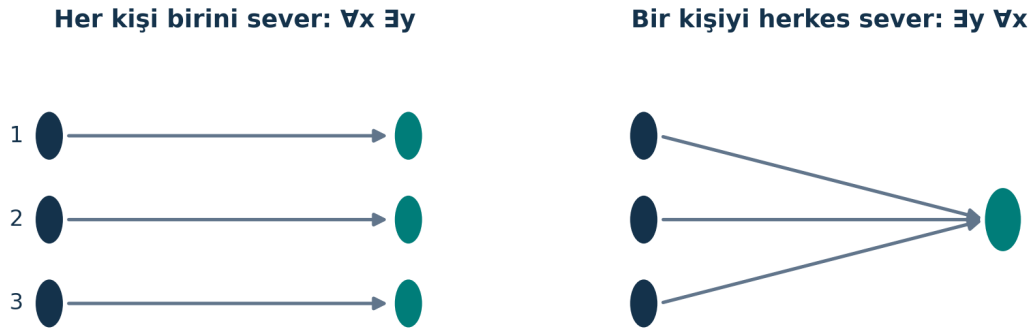
$$\forall x, y \ (Sibling(x, y) \Leftrightarrow Sibling(y, x)).$$

Farklı türden niceleyicilerin **sırası anlamı değiştirebilir**:

$\boxed{\forall x \ \exists y \ Loves(x, y)}$ "Herkes birini sever."

$\boxed{\exists y \ \forall x \ Loves(x, y)}$ "Herkesin sevdiği bir kişi vardır."

İlkinde her x için başka bir y seçilebilir; ikincisinde **aynı y** bütün x'ler için geçerli olmalıdır. Sonuç olarak ikinci ifade birinciden güçlüdür. Ebeveyn örneğiyle benzer ayrım: "Her öğrencinin bir danışmanı vardır" demek, "Bütün öğrencilerin danışmanı aynı kişidir" demek değildir.



Aynı niceleyiciler farklı sırada kullanıldığında ifadelerin anlamı değişebilir: her kişinin sevdiği ayrı kişiler ile herkesin sevdiği tek ortak kişi.

Aynı değişken isminin iç içe kullanımı kafa karıştırabilir:

$$\forall x \ (Crown(x) \vee (\exists x \ Brother(Richard, x))).$$

`Brother(Richard,x)` içindeki x, kendisine en yakın olan $\exists x$ tarafından bağlanır; dıştaki $\forall x$ bu kullanım üzerinde etkili değildir. İçteki x yerine z yazarsak anlam değişmez. Kitap bu nedenle farklı katmanlardaki niceleyicilere farklı değişken adları verir.

$\forall$ ile $\exists$ arasındaki bağlantılar

İki niceleyici olumsuzlama yoluyla birbirine dönüştürülebilir. "Herkes yaban havucunu sevmez" (kimse sevmez) ile "Yaban havucunu seven biri yoktur":

$$\forall x \ \neg Likes(x, Parsnips) \equiv \neg \exists x \ Likes(x, Parsnips).$$

"Herkes dondurma sever" ile "Dondurma sevmeyen biri yoktur":

$$\forall x \ Likes(x, IceCream) \equiv \neg \exists x \ \neg Likes(x, IceCream).$$

Genel kurallar:

$$\begin{aligned}\forall x\ \neg P(x) &\equiv \neg\exists x\ P(x),\\ \neg\forall x\ P(x) &\equiv \exists x\ \neg P(x),\\ \forall x\ P(x) &\equiv \neg\exists x\ \neg P(x),\\ \exists x\ P(x) &\equiv \neg\forall x\ \neg P(x).\end{aligned}$$

Bunlar niceleyicilere uyarlanmış **De Morgan kurallarıdır**; klasik bağlaçlarda da:

$$\begin{aligned}\neg(P \vee Q) &\equiv \neg P \wedge \neg Q,\\ \neg(P \wedge Q) &\equiv \neg P \vee \neg Q,\\ P \wedge Q &\equiv \neg(\neg P \vee \neg Q),\\ P \vee Q &\equiv \neg(\neg P \wedge \neg Q).\end{aligned}$$

$\forall$ evren elemanlarının hepsi üzerinde VE; $\exists$ ise en az biri üzerinde VEYA gibi davrandığından bu eşdeğerlikler doğaldır. Teknik olarak niceleyicilerden yalnızca biri seçilip diğeri tanımlanabilir; fakat okunabilirlik için ikisini de kullanırız.

Anlam farkına dikkat. $\neg\forall x(Student(x) \Rightarrow Successful(x))$, "Bütün öğrencilerin başarılı olduğu doğru değil" der: **En az bir başarısız öğrenci vardır.** Buna karşılık $\forall x(Student(x) \Rightarrow \neg Successful(x))$, "Hiçbir öğrenci başarılı değildir" demektir. Birincisi ikincisinden daha zayıftır.

8.2.7 Eşitlik (Equality)

Atomik cümleler yalnız yüklemlerle oluşturulmaz. İki terimin **aynı nesneye gönderme yaptığını** belirtmek için $=$ kullanılır:

$$Father(John) = Henry.$$

Bu cümle, John'un babasını gösteren terim ile Henry'yi gösteren sabitin aynı nesneye karşılık geldiğini ifade eder. Bir modelde eşitliğin doğru olması, iki terimin yorumlamada aynı nesneye eşlenmesine bağlıdır.

Eşitliğin değili **farklılık** ifade eder: $x \neq y$ ifadesi $\neg(x = y)$ için kısa gösterimdir. "Richard'ın en az iki kardeşi vardır":

$$\boxed{\exists x, y\ (Brother(x, Richard) \wedge Brother(y, Richard) \wedge x \neq y).}$$

Son koşul çıkarılırsa x ile y'nin ikisinin de John olduğu yorumlama ifadeyi doğru kılar; tek bir kardeş varlığı yanlışlıkla "iki kardeş" sayılmış olur. $x \neq y$ koşulu, **farklı iki kişinin** varlığını garanti eder.

8.2.8 Alternatif Bir Anlambilim? (An Alternative Semantics?)

Richard'ın kardeşlerinin **John ve Geoffrey** olduğunu varsayalım. Aşağıdaki iki olgu tek başına "Kardeşlerinin hepsi yalnız bu ikisidir" demeye yetmez:

$$Brother(John, Richard) \wedge Brother(Geoffrey, Richard).$$

Birinci eksik, `John` ve `Geoffrey` sabitlerinin aynı kişiyi gösterebilmesidir; $John \neq Geoffrey$ eklenmelidir. İkinci eksik, Richard'ın başka kardeşleri olabilmesidir. Kardeşlerin **tam olarak** bu iki kişi olduğunu söyleyen ifade şöyledir:

$$\begin{aligned} &Brother(John, Richard) \wedge Brother(Geoffrey, Richard) \\ &\wedge John \neq Geoffrey \\ &\wedge \forall x \ (Brother(x, Richard) \Rightarrow (x = John \vee x = Geoffrey)). \end{aligned}$$

Gündelik dilde kısa olan ifade standart FOL'de uzamaktadır; bu da çeviri hatalarına yol açabilir. Daha sezgisel bir anlam sistemi kurulabilir mi? Veritabanlarında yaygın olan yaklaşım üç varsayıma dayanır:

1. **Benzersiz isimler varsayımı (unique-names assumption):** Farklı sabit sembolleri farklı nesnelere gönderme yapar.
2. **Kapalı dünya varsayımı (closed-world assumption):** Doğru olduğu bilinmeyen atomik olgular yanlış kabul edilir.
3. **Evren kapanışı (domain closure):** Sabit sembolleriyle adlandırılanların dışında evren elemanı yoktur.

Bu üç koşulla tanımlanan **veritabanı anlambilimi (database semantics)** altında yukarıdaki iki kardeşlik olgusu yeterlidir: John ve Geoffrey ayrıdır, kaydedilmemiş başka kardeş yoktur. Mantıksal programlamada da benzer semantik yaklaşımlar kullanılabilir.

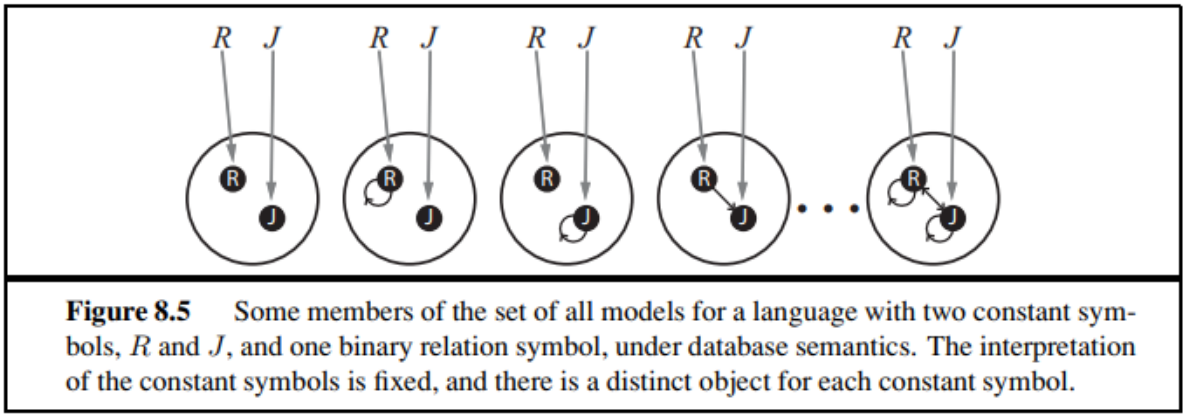


Figure 8.5 Some members of the set of all models for a language with two constant symbols, R and J, and one binary relation symbol, under database semantics. The interpretation of the constant symbols is fixed, and there is a distinct object for each constant symbol.

Şekil 8.5. Veritabanı anlambilimi altında iki farklı sabit R,J ve bir ikili ilişki için bazı modeller. Her modelde tam iki farklı nesne vardır.

Şekil 8.5'te iki sabit sembolü olduğundan **her modelde iki farklı nesne** vardır. İkili ilişkinin olası sıralı çiftleri (R, R), (R, J), (J, R), (J, J) olmak üzere dörttür. Her çift ilişkide var veya yok olabilir; dolayısıyla

$$\boxed{2^4 = 16 \text{ olası model}}$$

elde edilir. İlk modelde ilişki boş kümedir; diğerlerinde bir veya daha çok çift ilişkilidir; en kapsamlı modelde dört çiftin hepsi ilişkili olur. Standart FOL'deki **sonsuz sayıda mümkün modele** kıyasla bu büyük bir azalmadır; ancak dünyanın bütün nesneleri ve olguları hakkında daha kesin bilgi varsaymayı gerektirir.

Tek bir evrensel "en iyi" anlambilim yoktur. Hangi anlambilimin yararlı olduğu, bir alandaki bilginin ne kadar kısa ve sezgisel ifade edilebildiğine ve etkili çıkarım kuralları geliştirilip geliştirilemediğine bağlıdır. Veritabanı anlambilimi bütün nesneler ve olgular biliniyorsa

yararlıdır; belirsiz veya eksik bilgi altında zorlayıcı olabilir. Kaynağın kalanında standart FOL anlambilimi kullanılacaktır.

Özellik	Standart FOL	Veritabanı anlambilimi
İki isim aynı nesneyi gösterebilir mi?	Evet	Hayır
Bildirilmeyen atomik olgu yanlış mı?	Zorunlu değil	Evet
İsimsiz nesne bulunabilir mi?	Evet	Hayır
Uygunluk	Açık ve eksik bilgi	Kapalı ve kapsamlı kayıtlar

8.3 Birinci Dereceden Mantığın Kullanımı (Using First-Order Logic)

Artık ifade gücü yüksek bir mantık dili tanımlandığına göre, onu kullanmayı örneklerle öğrenebiliriz. Şimdiye kadarki kısa ifadeler yerine **basit problem alanlarını (domains)** sistematik biçimde temsil edeceğiz. Bilgi temsilindeki *domain*, hakkında bilgi söylemek istediğimiz dünya parçasıdır. Önce bilgi tabanına bilgi ekleme ve sorgulama arayüzü; sonra aile ilişkileri, sayılar, kümeler, listeler ve Wumpus dünyası ele alınacaktır. Bir sonraki bölüm daha kapsamlı bir elektronik devre örneğini işleyecektir.

8.3.1 Birinci Dereceden Mantıkta Bildirimler ve Sorgular

Önermeler mantığında olduğu gibi FOL'de de bilgi tabanına cümle eklemek için **TELL** kullanılır. Eklenen cümlelere **bildirimler (assertions)** denir. Örneğin John'un kral, Richard'ın insan olduğunu ve bütün kralların insan olduğunu bildirebiliriz:

```
TELL(KB, King(John))
TELL(KB, Person(Richard))
TELL(KB, ∀x (King(x) ⇒ Person(x)))
```

Bilgi tabanına soru sormak için **ASK** kullanılır:

```
ASK(KB, King(John))       -> True
ASK(KB, Person(John))     -> True
```

İkinci sorgu ayrıca bildirilmiş olmamasına rağmen **birinci ve üçüncü bildirimlerden mantıksal olarak çıktığı için** doğru cevaplanır. Sorgulara *queries* veya *goals* denir. Bilgi tabanından mantıksal olarak çıkan (entailed) bir sorguya olumlu cevap verilmelidir.

Niceleyicili bir sorgu da sorabiliriz:

```
ASK(KB, ∃x Person(x))     -> True
```

Ama "En az bir insan var mı?" sorusuna yalnızca "Evet" cevabı, "Saati söyleyebilir misin?" sorusuna "Evet" demek gibi yetersiz kalabilir. İnsan olan nesnelerin **hangileri** olduğunu bulmak için **ASKVARS** kullanılır:

```
ASKVARS(KB, Person(x))
```

Bu sorgu $\{x/John\}$ ve $\{x/Richard\}$ gibi cevaplar üretebilir. Bu sonuçlara **yerine koyma (substitution)** veya **bağlama listesi (binding list)** denir: $\{x/John\}$, sorgudaki x yerine John konulduğunu gösterir.

ASKVARS yaygın olarak yalnız **Horn cümlelerinden** oluşan bilgi tabanlarıyla ilişkilendirilir. Çünkü genel FOL'de varlığını bildiğimiz nesnenin hangisi olduğuna ilişkin kesin bir sabit belirleyemeyebiliriz. Örneğin bilgi tabanında yalnız

$$King(John) \vee King(Richard)$$

bulunuyorsa $\exists x\ King(x)$ doğrudur, fakat ne John'un ne Richard'ın kesin kral olduğu çıkar; dolayısıyla x'e ayrı bir kesin isim bağlanamaz.

Çözümlü sorgu. $Student(Ali)$ ve $\forall x(Student(x) \Rightarrow HasID(x))$ bilgilerinden $HasID(Ali)$ çıkar. Ancak $HasID(Mehmet)$ hakkında sonuç yoksa bunu **yanlış** sayamayız: Standart FOL'de çıkarılamayan bir olgu bilinmiyor olabilir.

8.3.2 Akrabalık Alanı (The Kinship Domain)

İlk alanımız **akrabalıktır**. "Elizabeth, Charles'ın annesidir", "Charles, William'ın babasıdır" gibi özel olguların yanında "Büyükanne bir ebeveynin annesidir" gibi genel kurallar vardır. Nesneler **insanlardır**. Tekli yüklemler $Male$ ve $Female$; ikili yüklemler $Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt, Uncle$ olarak seçilebilir. Kaynağın basitleştirilmiş modelinde herkesin birer annesi ve babası olduğu kabul edilerek $Mother$ ve $Father$ birer **fonksiyon** olarak kullanılır.

Şimdi bazı kavramları diğer kavramlarla tanımlayalım.

(1) Birinin annesi, onun kadın ebeveynidir:

$$\boxed{\forall m, c\ (Mother(c) = m \Leftrightarrow Female(m) \wedge Parent(m, c)).}$$

(2) Bir kişinin kocası, onun erkek eşidir:

$$\boxed{\forall w, h\ (Husband(h, w) \Leftrightarrow Male(h) \wedge Spouse(h, w)).}$$

(3) Erkek ve kadın kategorileri örnekte birbirini dışlayan ve tamamlayan iki kategoridir:

$$\boxed{\forall x\ (Male(x) \Leftrightarrow \neg Female(x)).}$$

Bu, kaynakta kullanılan bir **model varsayımıdır**, gerçek dünyadaki tüm cinsiyet kimliklerini kapsayan bir sınıflandırma değildir.

(4) Ebeveyn ve çocuk ters ilişkilerdir:

$$\boxed{\forall p, c\ (Parent(p, c) \Leftrightarrow Child(c, p)).}$$

(5) Birinin büyük ebeveyni, onun ebeveyninin ebeveynidir:

$$\boxed{\forall g, c\ (Grandparent(g, c) \Leftrightarrow \exists p\ (Parent(g, p) \wedge Parent(p, c))).}$$

(6) İki farklı kişi aynı ebeveynin çocuklarıysa kardeştir:

$$\boxed{\forall x, y\ (Sibling(x, y) \Leftrightarrow x \neq y \wedge \exists p\ (Parent(p, x) \wedge Parent(p, y))).}$$

Somut örnek. $Parent(Mehmet, Ali)$ ve $Parent(Mehmet, Ayse)$ ile $Ali \neq Ayse$ verildiyse $Sibling(Ali, Ayse)$ sonucuna varırız. $Parent(Fatma, Mehmet)$ ve $Parent(Mehmet, Ali)$ varsa $Grandparent(Fatma, Ali)$ çıkar.

Bu cümlelerin her biri akrabalık alanında birer **aksiyom** olarak kullanılabilir. Aksiyomlar yalnız matematiksel alanlara özgü değildir; belirli bir alanda hangi çıkarımların yapılabileceğini belirleyen temel bilgidir. Buradaki aksiyomlar aynı zamanda birer **tanımdır**: Bazı kavramları başka yüklemlerle tanımlarız. Örnekte tanımlar sonunda $Child$, $Spouse$ ve $Female$ gibi temel yüklemlere dayanır. Bir başka sistemde $Parent$, $Spouse$ ve $Male$ temel seçilebilirdi; tek bir zorunlu temel yüklem kümesi yoktur.

Aksiyom ve teorem arasındaki fark: Her mantıksal cümle aksiyom değildir. Aksiyomlardan mantıksal olarak çıkan cümle **teorem (theorem)** sayılır. Örneğin:

$$\forall x, y \ (Sibling(x, y) \Leftrightarrow Sibling(y, x))$$

kardeşliğin simetrik olduğunu belirtir. Bu sonuç, ortak ebeveyn ve $x \neq y$ içeren kardeş tanımından zaten çıkarılabilir; dolayısıyla ayrıca aksiyom olmasına gerek yoktur. Teorik olarak aksiyomlardan türetilebilen teoremleri saklamak yeni mantıksal sonuç yaratmaz; pratikte önceden bilinen sonuçları kullanmak çıkarım maliyetini azaltabilir.

Her aksiyom tam tanım değildir. $\forall x(Person(x) \Leftrightarrow \ldots)$ ifadesinin sağını eksiksiz yazmak zordur; bunun yerine kişi olmanın gerekli veya yeterli koşulları kısmen belirtilebilir. Aksiyom, $Male(Jim)$ ya da $Spouse(Jim, Laura)$ gibi doğrudan bir **olgu** da olabilir. Spesifik soruların cevapları, bu olgular ve genel kurallardan türetilen teoremlerdir.

Bazen beklenen sonuç çıkmıyorsa **eksik aksiyom** vardır. Örneğin $Spouse(Jim, Laura)$ ile $Jim \neq George$ bilgileri, birçok ülkede beklenen $\neg Spouse(George, Laura)$ sonucunu tek başına üretmez. Çünkü tek eşlilik koşulu tanımlanmamıştır. Basitleştirilmiş bir tek eşli evlilik modeli için eklenebilecek kural:

$$\forall x, y, z \ ((Spouse(x, y) \land Spouse(z, y)) \Rightarrow x = z).$$

Bu, eksik bilgiyle çalışan çıkarım sistemlerinin neden *dünya bilgisi* aksiyomlarına ihtiyaç duyduğunu gösterir.

8.3.3 Sayılar, Kümeler ve Listeler (Numbers, Sets, and Lists)

Doğal sayılar ve Peano aksiyomları

Sayılar, küçücük bir aksiyom çekirdeğinden geniş bir matematiksel teorinin kurulabileceğini gösterir. Burada doğal sayılar **negatif olmayan tam sayılar** olarak ele alınır. İhtiyacımız olanlar: `NatNum` yüklemi, `0` sabiti ve `S` **ardıl (successor)** fonksiyonu. Peano tarzı aksiyomlarla doğal sayılar özyinelemeli tanımlanır:

$$NatNum(0), \qquad \forall n \ (NatNum(n) \Rightarrow NatNum(S(n))).$$

Böylece $0, S(0), S(S(0)), \ldots$ doğal sayı olarak tanımlanır. Bu iki aksiyomun tek başına alışılmış doğal sayılar dışında başka elemanları da dışlamadığını, standart FOL anlambilimine ilişkin önceki bölümden görebiliriz. Ardıl fonksiyonu için ek kısıtlar:

$$\forall n \ (0 \neq S(n)), \qquad \forall m, n \ (m \neq n \Rightarrow S(m) \neq S(n)).$$

İlki sıfırın hiçbir nesnenin ardılı olmadığını, ikincisi farklı nesnelerin ardıllarının da farklı olduğunu söyler.

Toplama ardıl fonksiyonuyla iki aksiyomda tanımlanabilir:

$$\forall m \ (NatNum(m) \Rightarrow +(0, m) = m),$$

$$\forall m, n \ ((NatNum(m) \wedge NatNum(n)) \Rightarrow +(S(m), n) = S(+(m, n))).$$

Birinci aksiyom sıfırın eklenmesinin sayıyı değiştirmediğini; ikincisi toplamın ardıl üzerinden özyinelemeyle hesaplanacağını söyler. Kitapta `+(m,n)` gibi **prefix (önek)** gösterim kullanılır; günlük matematikteki $m + n$ ise **infix (ara-ek)** gösterimdir. $S(n)$ yerine $n + 1$ yazılırsa ikinci kural

$$(m + 1) + n = (m + n) + 1$$

şeklinde okunur. Anlamı değiştirmeden okunurluğu artıran bu tür kısaltmalara **syntactic sugar (sözdizimsel kolaylık)** denir. Çarpma tekrarlı toplama, üs alma tekrarlı çarpma ile tanımlanabilir; sonrasında bölme, kalan, asallık ve sayı teorisinin pek çok alanı geliştirilebilir. Kaynak, bu anlatımın çekirdeğinde bir sabit, bir fonksiyon, bir yüklem ve dört temel aksiyom bulunduğunu vurgular. Peano aritmetiğinin tüm kapsamı için **tümevarım ilkesi** de önemlidir; ikinci dereceden mantıkla ilişkisi 9. Bölüm'de ele alınır.

Adım adım toplama: $2 = S(S(0))$, $1 = S(0)$ yazalım. O halde $2 + 1 = S(S(0)) + S(0) = S(S(0) + S(0)) = S(S(0 + S(0))) = S(S(S(0))) = 3$. Sonucu programın aritmetik yordamına değil, tanımlanan ardıl ve toplama aksiyomlarına dayandırdık.

Kümeler

Kümeler hem matematikte hem günlük bilgi temsilinde önemlidir; sayı teorisi dahi küme teorisiyle kurulabilir. Boş küme, bir elemana sahip olma, iki kümenin birleşimi ve kesişimi, alt küme olma gibi kavramları temsil etmek isteriz. Standart matematiksel semboller okunabilirlik için kullanılır:

Gösterim	Anlamı
$\{\}$	Boş küme sabiti
$Set(s)$	s bir kümedir (tekli yüklem)
$x \in s$	x, s kümesinin üyesidir (ikili yüklem)
$s_1 \subseteq s_2$	s1, s2'nin alt kümesidir (ikili yüklem)
$s_1 \cap s_2$	Kesişim (ikili fonksiyon)
$s_1 \cup s_2$	Birleşim (ikili fonksiyon)
$\{x \mid s\}$	s kümesine x'in eklenmesiyle oluşan küme (ikili fonksiyon)

Kaynakta verilen olası bir küme aksiyomları grubu **sekiz** koşul içerir:

1. Tek kümeler boş küme ile başka kümeye eleman eklenerek elde edilen kümelerdir:

$$\forall s \ (Set(s) \Leftrightarrow (s = \{\}) \vee \exists x, s_2 \ (Set(s_2) \wedge s = \{x \mid s_2\})).$$

2. Boş küme hiçbir biçimde bir elemana ekleme işleminin sonucu olamaz:

$$\neg\exists x, s\ (\{x \mid s\} = \{\}).$$

3. Zaten içerilen elemanın tekrar eklenmesi kümeyi değiştirmez:

$$\forall x, s\ (x \in s \Leftrightarrow s = \{x \mid s\}).$$

4. Kümeye üyelik özyinelemeyle tanımlanır: x, son eklenen y'ye eşit olabilir veya daha önceki s2 kümesinde bulunabilir:

$$\forall x, s\ (x \in s \Leftrightarrow \exists y, s_2\ (s = \{y \mid s_2\} \wedge (x = y \vee x \in s_2))).$$

5. Alt küme: Birinci kümenin her üyesi ikinci kümenin de üyesiyse:

$$\forall s_1, s_2\ (s_1 \subseteq s_2 \Leftrightarrow \forall x\ (x \in s_1 \Rightarrow x \in s_2)).$$

6. İki küme ancak karşılıklı alt kümeyse eşittir:

$$\forall s_1, s_2\ (s_1 = s_2 \Leftrightarrow (s_1 \subseteq s_2 \wedge s_2 \subseteq s_1)).$$

7. Kesişimde üyelik, iki kümede birden üyeliktir:

$$\forall x, s_1, s_2\ (x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)).$$

8. Birleşimde üyelik, kümelerden en az birinde üyeliktir:

$$\forall x, s_1, s_2\ (x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)).$$

Çözümlü küme örneği. $A = \{1, 2, 3\}$ ve $B = \{3, 4, 5\}$ olsun. O halde $A \cup B = \{1, 2, 3, 4, 5\}$ ve $A \cap B = \{3\}$. $3 \in A \cap B$ doğrudur çünkü $3 \in A$ ve $3 \in B$ doğrudur. Fakat $2 \in A \cap B$ yanlıştır; 2, B'de yoktur.

Listeler

Listeler kümelere benzer; iki farkları vardır: **sıralıdırlar** ve aynı eleman birden çok kez bulunabilir. Lisp gösterimini örnek alırız: `Nil` boş listenin sabitidir; `Cons`, `Append`, `First` ve `Rest` fonksiyonlardır; `Find` listede üyeliğe ilişkin yüklemdir; `List?` yalnız listeler için doğru olan yüklemdir.

Boş liste `[]` ile, $Cons(x, Nil)$ tek elemanlı `[x]` ile, $Cons(x, y)$ başa eleman ekleme işlemi `[x|y]` ile yazılabilir. Üçlü liste:

$$[A, B, C] = Cons(A, Cons(B, Cons(C, Nil))).$$

Kitap 8.16 numaralı alıştırmada okurun liste aksiyomlarını geliştirmesini ister. Kümeyle karşılaştırma: $\{A, B, A\} = \{A, B\}$ iken `[A,B,A]` ile `[A,B]` aynı liste değildir. `[A,B]` ile `[B,A]` da farklı listelerdir.

8.3.4 Wumpus Dünyası (The Wumpus World)

7. Bölüm'de aynı dünyayı önermeler mantığıyla anlatmak için çok sayıda kare ve zaman kuralı gerekiyordu. FOL ile istenen bilgiler çok daha **kısa ve doğal** biçimde tanımlanabilir. Ajanın her algısında beş bileşen bulunur. Bilgi tabanına **algının gerçekleştiği zamanı** da eklemeliyiz; yoksa ajan neyi ne zaman gördüğünü karıştırır. Zaman basamakları tam sayılarla gösterilir.

Örnek bir algı kaydı:

$$Percept([Stench, Breeze, Glitter, None, None], 5).$$

Burada `Percept` ikili bir yüklemdir; ilk argümanı koku, esinti, parıltı ve diğer algıları içeren listedir, ikincisi **5. zaman adımıdır**. Ajanın eylemleri terim olarak yazılabilir: `Turn(Right)`, `Turn(Left)`, `Forward`, `Shoot`, `Grab`, `Climb`.

En iyi eylemi bulmak için kitapta:

$$ASKVARS(\exists a\ BestAction(a, 5))$$

biçiminde bir sorgu verilir. Sonuç $\{a/Grab\}$ olursa ajan **Grab** eylemini seçer. `ASKVARS` burada uygun eylem için değişken bağlaması veren bir sorgu yordamını temsil eder.

Algıdan olgu çıkarma kuralları:

$$\forall t, s, g, m, c\ (Percept([s, Breeze, g, m, c], t) \Rightarrow Breeze(t)),$$
$$\forall t, s, b, m, c\ (Percept([s, b, Glitter, m, c], t) \Rightarrow Glitter(t)).$$

Birinci kural, algı listesinin ikinci bileşeni esintiyse o anda esinti algılandığını; ikinci kural, üçüncü bileşen parıltıysa o anda parıltı algılandığını belirtir. Bu, kitapta daha sonra ayrıntılandırılan **algılama (perception)** sürecinin basit bir mantıksal örneğidir. Her kuraldaki zaman nicelemesi önemlidir; önermeler mantığında her zaman basamağı için ayrı kopyalar gerekirdi.

Basit refleks kuralı:

$$\boxed{\forall t\ (Glitter(t) \Rightarrow BestAction(Grab, t)).}$$

Beşinci adımda parıltı varsa $BestAction(Grab, 5)$ çıkarılabilir.

Şimdi ortamı modelleyelim. Kareler, çukurlar ve Wumpus olası nesnelerdir. Her kareye `Square1,2` gibi özel sabit verebiliriz; fakat komşuluğu her çift için ayrı kaydetmek gerekir. Bunun yerine koordinatları iki elemanlı bir liste terimiyle temsil etmek daha uygundur: `[1,2]`.

İki karenin komşuluğu:

$$\begin{aligned}\forall x, y, a, b\ Adjacent([x, y], [a, b]) \Leftrightarrow\\ (x = a \wedge (y = b - 1 \vee y = b + 1))\\ \vee (y = b \wedge (x = a - 1 \vee x = a + 1)).\end{aligned}$$

Bu kural aynı satırda yan yana veya aynı sütunda alt alta kareleri komşu kabul eder. Çukurları ayrı nesneler olarak adlandırmaya gerek yoktur; kareye özgü $Pit(s)$ yüklemi kullanılır. Modelde **tek bir** Wumpus bulunduğundan `Wumpus` sabit sembolü uygundur.

Ajan hareket edebildiği için konumu zamana bağlı ifade edilir:

$$At(Agent, s, t).$$

Wumpus'un yerini sabitlemek için $\forall t \;\; At(Wumpus, [2,2], t)$ yazılabilir. Bir nesne belirli bir anda iki farklı yerde birden bulunamaz:

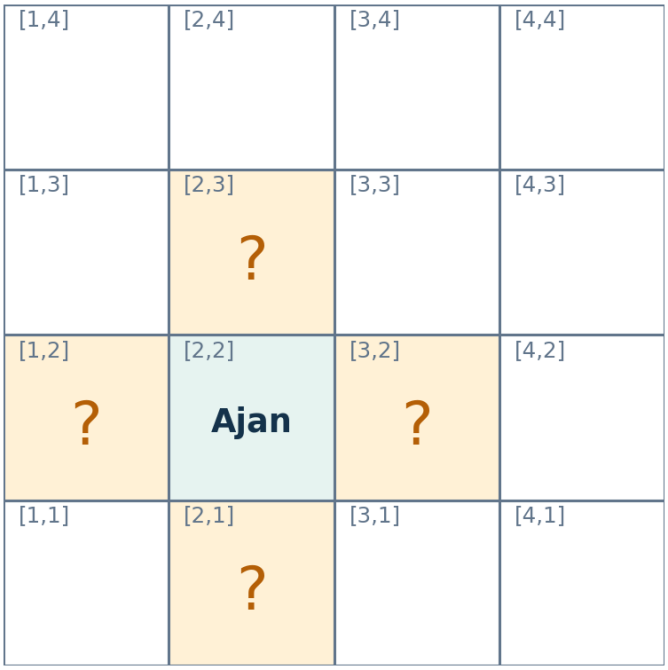
$$\forall x, s_1, s_2, t \; ((At(x, s_1, t) \wedge At(x, s_2, t)) \Rightarrow s_1 = s_2).$$

Bir ajan s karesindeyken t anında esinti algılarsa s karesi esintilidir:

$$\boxed{\forall s, t \; ((At(Agent, s, t) \wedge Breeze(t)) \Rightarrow Breezy(s)).}$$

$Breezy$ yükleminde **zaman yoktur**; çünkü çukurlar hareket etmez, dolayısıyla bir karenin esintili olma özelliği sabittir. Yerlerin esintili veya esintisiz olduğu bilgileri çukurların yerini belirlemeyi sağlar. Bütün dünya için tek bir genel kural yeterlidir:

$$\boxed{\forall s \; (Breezy(s) \Leftrightarrow \exists r \; (Adjacent(r, s) \wedge Pit(r))).} \tag{8.4}$$



Örnek 4x4 Wumpus ızgarası. Ortadaki [2,2] karesinde esinti algılanırsa sarı komşulardan en az birinde çukur olabilir. Soru işaretleri kesin konum belirtmez.

Adım adım örnek: $At(Agent, [2,2], 5)$ ve $Breeze(5)$ olsun. Algı–konum kuralıyla $Breezy([2,2])$ çıkar. (8.4) ile [2,2]'nin komşularından **en az birinde** çukur bulunduğu çıkar. Olası kareler [1,2], [3,2], [2,1], [2,3] olup tek bir kare kesinleşmez. Bir komşuda esinti *olmaması* gibi ek bilgi geldiğinde seçenekler daraltılabilir.

FOL'de zaman üzerinde niceleme yapılabildiğinden **her yüklem için tek bir ardıl durum aksiyomu (successor-state axiom)** kullanılabilir. Örneğin okun varlığını güncelleyen kural:

$$\boxed{\forall t \; (HaveArrow(t+1) \Leftrightarrow (HaveArrow(t) \wedge \neg Action(Shoot, t))).}$$

Ajanın sonraki adımda oku vardır ancak ve ancak önceki anda oku varsa ve önceki adımda ateş etmemişse. Ajanın konumu ve yönelimi için de uzay ve zaman üzerinde niceleme yapan benzer kurallar yazılabilir. Mantıksal çıkarım sayesinde doğrudan gözlemlenmeyen dünya özellikleri takip edilir; bu tür kuralların planlamadaki rolü 10. Bölüm'de ayrıntılandırılır.

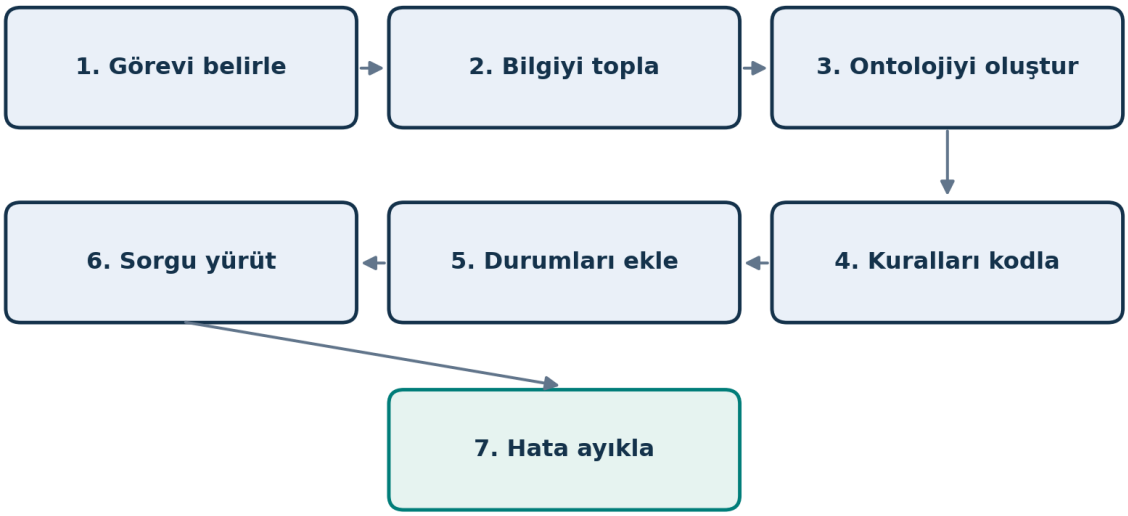
8.4 Birinci Dereceden Mantıkta Bilgi Mühendisliği

Önceki bölümde FOL'nin birkaç basit problem alanındaki kullanımı gösterildi. Şimdi bilgi tabanının tasarlanıp geliştirilmesi için izlenen genel yöntemi inceleyeceğiz. Bu sürece **bilgi mühendisliği (knowledge engineering)** denir. **Bilgi mühendisi**, bir problem alanını araştırır, önemli kavramları öğrenir ve o alandaki nesneler ile ilişkiler için biçimsel bir temsil üretir.

Aşağıda bilgi mühendisliğini tanıdık bir problem olan **elektronik devreler** üzerinden örnekleyeceğiz. Yaklaşım, alanı açıkça sınırlandırılmış ve sorulabilecek sorguların türü önceden bilinen **özel amaçlı bilgi tabanları** için uygundur. İnsan bilgisinin çok büyük bir bölümünü kapsamaya ve doğal dil anlamaya çalışan genel bilgi tabanları 12. Bölüm'de ele alınır.

8.4.1 Bilgi Mühendisliği Süreci (The Knowledge-Engineering Process)

Projelerin kapsamı, içeriği ve zorluğu farklı olsa da **yedi temel adım** her projede yer alır.



Bilgi mühendisliğinin yedi aşaması: görev, bilgi edinimi, ontoloji, genel aksiyomlar, probleme özgü olgular, sorgular ve hata ayıklama. Açıklayıcı ek çizim.

1. Görevi belirleme (Identify the Task)

Bilgi mühendisi, bilgi tabanının hangi **soruları cevaplayacağını** ve her somut problem için ne tür **olguların sağlanacağını** sınırlandırır. Örneğin Wumpus bilgi tabanı eylem mi seçmeli, yoksa yalnız ortam içeriğine ilişkin soruları mı cevaplamalı? Algı verileri ajanın mevcut

konumunu içerecek mi? Görev, olaylardan cevaplara ulaşmak için hangi bilgilerin gerekli olduğunu belirler. Bu süreç 2. Bölüm'deki **PEAS** ajan tasarımına benzer.

2. İlgili bilgileri toplama (Assemble the Relevant Knowledge)

Bilgi mühendisi alan uzmanı olabilir veya uzmanlardan bilgi edinmesi gerekebilir; buna **knowledge acquisition (bilgi edinimi)** denir. Bu aşamada bilgiler henüz mantıksal sembollere çevrilmez. Amaç alanın gerçekten nasıl işlediğini ve hangi bilginin görev açısından ilgili olduğunu anlamaktır. Wumpus dünyası yapay ve açık kurallarla tanımlandığından nispeten kolaydır; ancak komşuluğun tanımı bile ayrıca türetilmiştir. Gerçek dünyada neyin ilgili olduğu daha zordur: VLSI simülasyonunda kaçak kapasitans veya deri etkisi belirli amaçlarda gerekli, diğerlerinde gereksiz olabilir.

3. Yüklem, fonksiyon ve sabit sözlüğünü seçme (Decide on a Vocabulary)

Önemli alan kavramları mantıksal isimlere çevrilir. Çukurlar nesne mi, karelerin özelliği mi? Ajanın yönü yüklem mi, fonksiyon mu? Wumpus'un yeri zamana bağlı mı? Bunlar bilgi mühendisliğinde bir çeşit **tasarım/üslup kararlarıdır** ve tıpkı programlama üslubu gibi başarımı etkileyebilir. Seçilen sembol dağarcığı, alanın **ontolojisi (ontology)** olarak adlandırılır. Ontoloji hangi *tür şeylerin* var olduğunu belirler; onların hangi bireysel özellikleri ve bağlantıları taşıdığını henüz belirlemez.

4. Genel alan bilgisini kodlama (Encode General Knowledge)

Sözlükteki terimler için **aksiyomlar** yazılır; böylece mümkün olduğunca açık anlamlar belirlenir ve alan uzmanı bilgiyi kontrol edebilir. Bu aşama, önceki kavram seçimlerinde yanılgıları veya eksikleri gösterebilir; dolayısıyla üçüncü adıma dönülüp süreç tekrarlanabilir.

5. Somut problem durumunu kodlama (Encode the Specific Instance)

Ontoloji özenle tasarlanmışsa özel durumu girmek kolaydır: Önceden tanımlanmış kavramlara ilişkin **atomik olgular** eklenir. Bir ajan bunları sensörlerinden alır; bağımsız bilgi tabanına ise bir programa girdi verilmesi gibi ek cümleler sağlanır.

6. Sorgulama ve cevap alma (Pose Queries)

Çıkarım işlemi, genel aksiyomlar ile probleme özgü olguları kullanarak ilgilendiğimiz sonuçları türetir. Böylece her özel durum için ayrı bir çözüm algoritması yazma ihtiyacını azaltırız.

7. Bilgi tabanını hata ayıklama (Debug the Knowledge Base)

İlk denemelerde cevaplar beklenen cevaplar olmayabilir. Sağlam bir çıkarım yöntemi **yazılan aksiyomlar bakımından doğru** cevap verir; sorun çoğu kez aksiyomların eksik, fazla zayıf veya hatalı olmasıdır. Örneğin Wumpus için yalnız:

$$\forall s \; (Smelly(s) \Rightarrow Adjacent(Home(Wumpus), s))$$

kuralı verilirse kötü kokulu bir karenin Wumpus'a komşu olduğu çıkarılabilir; fakat ters yöndeki bilgiye ihtiyaç duyulan **Wumpus yokluğu ispatları** gerçekleştirilemez. Tam koşul gerektiğinde çift yönlü $\Leftrightarrow$ seçilmelidir.

Yanlış bir aksiyom başka bir hata kaynağıdır:

$$\forall x \; (NumOfLegs(x, 4) \Rightarrow Mammal(x)).$$

"Dört bacağı olan her şey memelidir" ifadesi sürüngenler, amfibiler ve özellikle dört ayaklı masalar nedeniyle **yanlıştır**. Bu yanlışlık, bilgi tabanının gerisini görmeden gerçek dünyaya bakılarak anlaşılabilir. Sıradan programdaki `offset = position + 1` satırının doğru olup olmadığını ise çevresindeki kod ve değişkenlerin kullanım amacı görülmeden belirlemek kolay değildir.

Yedi adımın mantığı: (1) Neyi çözüyorsun? (2) Hangi alan bilgisini biliyorsun? (3) Hangi kavramları temsil ediyorsun? (4) Genel kuralların neler? (5) Bu örneğin gerçekleri neler? (6) Hangi sonucu soruyorsun? (7) Beklenen cevap çıkmazsa nerede hata var?

8.4.2 Elektronik Devreler Alanı (The Electronic Circuits Domain)

Şimdi Şekil 8.6'daki dijital devre için bir ontoloji ve bilgi tabanı geliştireceğiz. Devrenin bir **bir bitlik tam toplayıcı (one-bit full adder)** olduğu ileri sürülmektedir; mantıksal temsil sayesinde bu iddiayı doğrulamayı amaçlıyoruz.

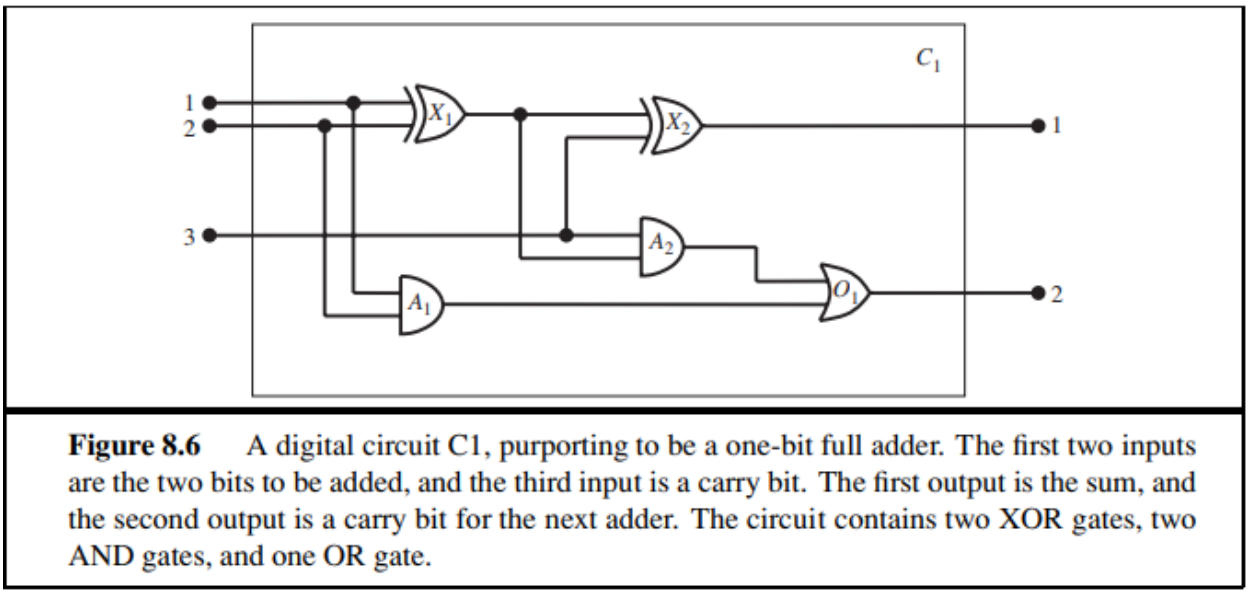


Figure 8.6 A digital circuit C1, purporting to be a one-bit full adder. The first two inputs are the two bits to be added, and the third input is a carry bit. The first output is the sum, and the second output is a carry bit for the next adder. The circuit contains two XOR gates, two AND gates, and one OR gate.

Şekil 8.6. C1 dijital devresi: ilk iki giriş toplanacak bitler, üçüncü giriş elde biti; ilk çıkış toplam, ikinci çıkış sonraki toplayıcıya taşınan elde bitidir. İki XOR (X1, X2), iki AND (A1, A2) ve bir OR (O1) kapısı vardır. Orijinal şekil kullanıcı tarafından sağlanmıştır.

Adım 1 — Görevi belirleme

Dijital devrelerle ilgili en üst düzey sorulardan biri **işlevsellik** sorusudur: Şekil 8.6 gerçekten toplama yapıyor mu? Bütün girdiler 1 ise A2 kapısının çıktısı ne olur? Devrenin **yapısı** da sorgulanabilir: İlk giriş terminaline bağlanan bütün kapılar nelerdir? Geri besleme döngüsü var mı?

Bu bölümde amacımız **işlevsellik ve bağlantıları** incelemektir. Gerçek devrelerin zaman gecikmesi, alan, güç tüketimi ve üretim maliyeti gibi ileri düzey değerlendirmeleri ise fazladan bilgi gerektirir.

Adım 2 — İlgili bilgileri toplama

Dijital devreler **kablolar** ve **kapılardan** oluşur. Sinyaller kablolardan geçerek kapı girişlerine ulaşır; kapı, giriş sinyallerini kendi mantıksal işlevine göre dönüştürür ve çıkışındaki sinyal başka terminale iletilir. Bu örnekte dört tür kapı vardır: **AND, OR, XOR, NOT**. AND, OR ve XOR'un **iki**, NOT'un **bir** giriş terminali vardır; hepsinin **bir** çıkışı bulunur. Kapılar gibi devrelerin de giriş ve çıkış terminalleri vardır.

Kapı düzeyinde doğru davranışı ve bağlantıları incelemek için kabloların fiziksel güzergâhlarını ve birleşme noktalarını ayrı ayrı temsil etmemiz gerekmez. Bir çıkış terminalinin başka giriş terminaline bağlı olduğunu söylemek yeterlidir. Boyut, biçim, renk ve maliyet de bu sorular açısından ilgisizdir. Fakat arızalı bir kabloyu teşhis edeceksek kablolar, zamanlama hatalarını araştıracaksak gecikmeler, ekonomik tasarım yapacaksak maliyet ve rakiplere göre hız ontolojiye eklenmelidir.

Adım 3 — Sembol dağarcığını belirleme

Kapılar, devreler, terminaller, sinyaller ve bağlantıları temsil etmeliyiz. Her kapı ayrı bir nesnedir; $Gate(X1)$ X1'in kapı olduğunu, $Type(X1) = XOR$ onun XOR türünde olduğunu bildirir. Kapının yalnızca **bir türü** bulunduğundan tür bilgisi fonksiyonla temsil edilir. Benzer şekilde $Circuit(C1)$ C1'in devre olduğunu belirtir.

$Terminal(t)$ bir terminali ifade eder; $In(1, X1)$ X1 kapısının **birinci giriş terminali**, $Out(1, X1)$ **birinci çıkış terminalidir**. $Arity(c, i, j)$ devre c'nin **i giriş** ve **j çıkış** terminaline sahip olduğunu belirtir. Kitapta `Arity` hakkında bir yerde fonksiyon sözcüğü kullanılsa da gösterimi $Arity(c, i, j)$ bir **yüklem** biçimindedir; not boyunca bunu yüklem kabul ediyoruz.

$Connected(t_1, t_2)$ iki terminalin bağlantısını ifade eder:

$$Connected(Out(1, X1), In(1, X2)).$$

Son olarak her terminaldeki sinyali temsil etmeliyiz. $On(t)$ tekli yüklemi kullanılabilir; fakat "Çıkış terminallerinde hangi değerler bulunur?" gibi sorgularda **0 ve 1'i nesne** kabul edip $Signal(t)$ fonksiyonuyla terminal değerini göstermek daha elverişlidir:

$$Signal(In(1, X1)) = 1.$$

Sembol	Rolü	Açıklaması
$Gate(g)$	Yüklem	g bir kapıdır
$Circuit(c)$	Yüklem	c bir devredir
$Terminal(t)$	Yüklem	t bir terminaldir
$Type(g)$	Fonksiyon	Kapının türü (AND/OR/XOR/NOT)
$In(n, g)$ ve $Out(n, g)$	Fonksiyon	n'inci giriş / çıkış terminali
$Arity(c, i, j)$	Yüklem	Giriş ve çıkış terminali sayıları
$Connected(t_1, t_2)$	Yüklem	İki terminal bağlantılıdır
$Signal(t)$	Fonksiyon	Terminal sinyal değeri (0/1)

Adım 4 — Genel alan aksiyomlarını kodlama

İyi ontolojinin bir göstergesi, kısa ve açık kurallarla çalışabilmesidir. Kaynakta **12 aksiyom grubu** kullanılır.

Aksiyom 1: Bağlı terminaller aynı sinyali taşır.

$$\forall t_1, t_2 \; (Terminal(t_1) \wedge Terminal(t_2) \\ \wedge Connected(t_1, t_2) \Rightarrow Signal(t_1) = Signal(t_2)).$$

Aksiyom 2: Her terminal sinyali 0 veya 1'dir.

$$\forall t \; (Terminal(t) \Rightarrow (Signal(t) = 1 \vee Signal(t) = 0)).$$

Aksiyom 3: Bağlantı simetriktir.

$$\forall t_1, t_2 \; (Connected(t_1, t_2) \Leftrightarrow Connected(t_2, t_1)).$$

Bu **bağlantının** simetrik olduğunu söyler; elektriksel sinyalin ters yönde aktif olarak iletildiğini söylemez.

Aksiyom 4: Kapı türü dört seçenekten biridir.

$$\forall g, k \; (Gate(g) \wedge k = Type(g) \Rightarrow \\ (k = AND \vee k = OR \vee k = XOR \vee k = NOT)).$$

Aksiyom 5: AND çıktısı 0 ancak ve ancak girdilerden en az biri 0 ise.

$$\forall g \; (Gate(g) \wedge Type(g) = AND \Rightarrow \\ (Signal(Out(1, g)) = 0 \Leftrightarrow \\ \exists n \; Signal(In(n, g)) = 0)).$$

Aksiyom 6: OR çıktısı 1 ancak ve ancak girdilerden en az biri 1 ise.

$$\forall g \; (Gate(g) \wedge Type(g) = OR \Rightarrow \\ (Signal(Out(1, g)) = 1 \Leftrightarrow \\ \exists n \; Signal(In(n, g)) = 1)).$$

Aksiyom 7: XOR çıktısı 1 ancak ve ancak iki girdi farklıysa.

$$\forall g \; (Gate(g) \wedge Type(g) = XOR \Rightarrow \\ (Signal(Out(1, g)) = 1 \Leftrightarrow \\ Signal(In(1, g)) \neq Signal(In(2, g)))).$$

Aksiyom 8: NOT çıktısı girdisinden farklıdır.

$$\forall g \; (Gate(g) \wedge Type(g) = NOT \Rightarrow \\ Signal(Out(1, g)) \neq Signal(In(1, g))).$$

Aksiyom 9: Kapıların giriş ve çıkış sayıları.

$$\forall g \; ((Gate(g) \wedge Type(g) = NOT) \Rightarrow Arity(g, 1, 1)),$$

$$\forall g, k\ (Gate(g) \land k = Type(g) \land \\ (k = AND \lor k = OR \lor k = XOR) \Rightarrow Arity(g, 2, 1)).$$

Aksiyom 10: Devrede, belirtilen giriş ve çıkış sayısı kadar terminal vardır; bu sayının ötesinde `Nothing` bulunur. Kaynağın biçimsel kuralını, geri kalan sayfalarla uyumlu $In(n, c)$ argüman sırasıyla yazarsak:

$$\begin{aligned}\forall c, i, j\ (Circuit(c) \land Arity(c, i, j)) \Rightarrow \\ \forall n\ ((n \leq i \Rightarrow Terminal(In(n, c))) \\ \land (n > i \Rightarrow In(n, c) = Nothing)) \\ \land \forall n\ ((n \leq j \Rightarrow Terminal(Out(n, c))) \\ \land (n > j \Rightarrow Out(n, c) = Nothing)).\end{aligned}$$

Kaynak sayfada bu özel aksiyomda argümanların `In(c,n)` olarak yazıldığı, örnek bağlantılarda ise `In(n,c)` biçiminin kullanıldığı görülür; burada **indeks önce** gelmektedir. Matematiksel olarak yalnız **geçerli terminal indeksleri** kastedilir; sıra ve başlangıç indeksleri alanın tanımlarıyla belirlenir.

Aksiyom 11: Kapılar, terminaller, sinyal değerleri, kapı türleri ve `Nothing` ayrı tür nesnelerdir. Kaynağın sıkıştırılmış gösterimindeki amaç, birbirine karışmaması gereken türlerin farklılığını ifade etmektir. Örneğin:

$$\forall g, t\ ((Gate(g) \land Terminal(t)) \Rightarrow g \neq t),$$

$$1 \neq 0, \qquad AND \neq OR, \qquad XOR \neq NOT.$$

Kapı/terminal nesneleri ile 0, 1, AND, OR, XOR, NOT, Nothing arasında gerekli tüm farklılıkların da tanımlanması gerekir. Kaynaktaki zincirleme $g \neq t \neq 1 \neq 0 \neq \cdots$ kısa bir anlatım olarak görünür; biçimsel FOL'de **bütün ilgili çiftler için** fark ayrı ayrı veya şema halinde verilmelidir.

Aksiyom 12: Her kapı bir devredir.

$$\boxed{\forall g\ (Gate(g) \Rightarrow Circuit(g)).}$$

Bu, birden çok kapıyı içeren daha büyük devrelerin yapıtaşı olarak kapıların kullanılmasını sağlar.

Mantık kapılarını hatırlama

A	B	$A \land B$ (AND)	$A \lor B$ (OR)	$A \oplus B$ (XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

AND iki giriş de 1 ise 1, OR en az biri 1 ise 1, XOR girişler farklıysa 1 üretir. NOT ise tek girişin tersini üretir.

Adım 5 — Şekil 8.6'nın özel durumunu kodlama

Şekildeki bütün devre **C1** adını alır; üç girişi ve iki çıkışı vardır:

$$Circuit(C1) \wedge Arity(C1, 3, 2).$$

İçindeki beş kapı:

$$\begin{aligned} Gate(X1) \wedge Type(X1) &= XOR, \\ Gate(X2) \wedge Type(X2) &= XOR, \\ Gate(A1) \wedge Type(A1) &= AND, \\ Gate(A2) \wedge Type(A2) &= AND, \\ Gate(O1) \wedge Type(O1) &= OR. \end{aligned}$$

Bağlantılar da özel problem olguları olarak saklanır. Kaynaktaki 12 bağlantının tamamı aşağıdadır:

```
Connected(Out(1,X1), In(1,X2))
Connected(Out(1,X1), In(2,A2))
Connected(Out(1,A2), In(1,O1))
Connected(Out(1,A1), In(2,O1))
Connected(Out(1,X2), Out(1,C1))
Connected(Out(1,O1), Out(2,C1))

Connected(In(1,C1), In(1,X1))
Connected(In(1,C1), In(1,A1))
Connected(In(2,C1), In(2,X1))
Connected(In(2,C1), In(2,A1))
Connected(In(3,C1), In(2,X2))
Connected(In(3,C1), In(1,A2))
```

Bu bağlantılar şeklin çizgilerinin izlediği fiziksel güzergâha değil **hangi terminallerin bağlı olduğuna** odaklanır. İlk iki giriş bitini A ve B, üçüncü girişten gelen eldeyi C_{in} olarak adlandırırsak:

$$\begin{aligned} X1 &= A \oplus B, \\ A1 &= A \wedge B, \\ X2 &= X1 \oplus C_{in}, \\ A2 &= X1 \wedge C_{in}, \\ O1 &= A1 \vee A2. \end{aligned}$$

Devrenin toplam çıkışı X2, elde çıkışı O1 olduğuna göre:

$$\boxed{S = A \oplus B \oplus C_{in}}$$

$$\boxed{C_{out} = (A \wedge B) \vee ((A \oplus B) \wedge C_{in}).}$$

Tam toplayıcı (full adder) üç adet tek bitlik girdiyi toplar; S en düşük anlamlı toplam biti, C_{out} sonraki basamağa aktarılan elde bitidir.

Adım adım tam toplayıcı örneği

$A = 1, B = 0, C_{in} = 1$ olsun.

1. İlk XOR: $X1 = 1 \oplus 0 = 1$.
2. İlk AND: $A1 = 1 \wedge 0 = 0$.
3. İkinci XOR: $X2 = 1 \oplus 1 = 0$; **toplam biti** $S = 0$.
4. İkinci AND: $A2 = 1 \wedge 1 = 1$.
5. OR: $O1 = 0 \vee 1 = 1$; **elde biti** $C_{out} = 1$.

Böylece sonuç $C_{out}S = 10_2 = 2$ elde edilir; gerçekten $1 + 0 + 1 = 2$'dir. Kapı düzeyindeki çözümle matematiksel toplam uyumludur.

Bütün girişlerin doğruluk tablosu

Üç bitlik giriş $2^3 = 8$ farklı kombinasyon üretir. Bütün kapıların aradeğerleri ve çıktıları:

A	B	C_{in}	X1	A1	X2 = S	A2	O1 = C_{out}
0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	1	0	0	1	1
1	0	0	1	0	1	0	0
1	0	1	1	0	0	1	1
1	1	0	0	1	0	0	1
1	1	1	0	1	1	0	1

Her satırda $A + B + C_{in} = 2C_{out} + S$ eşitliği sağlanır. Bu sonuç, C1'in **tam toplayıcı** davranışını doğrular.

Adım 6 — Sorgular sorarak devreyi doğrulama

İlk soru: **Hangi giriş kombinasyonlarında C1'in ilk çıkışı (toplam) 0, ikinci çıkışı (elde) 1 olur?**

$$\begin{aligned}\exists i_1, i_2, i_3\ (&Signal(In(1, C1)) = i_1 \\ &\wedge Signal(In(2, C1)) = i_2 \\ &\wedge Signal(In(3, C1)) = i_3 \\ &\wedge Signal(Out(1, C1)) = 0 \\ &\wedge Signal(Out(2, C1)) = 1).\end{aligned}$$

Bu cümleyi doğru yapan üç bağlama vardır:

```
{i1/1, i2/1, i3/0}
{i1/1, i2/0, i3/1}
{i1/0, i2/1, i3/1}
```

Her birinde girişlerin toplamı 2 olur ve ikili toplam **10**'dur. `ASKVARS` benzeri bir çıkarım sorgusu bu üç çözümü üretir.

İkinci soru: **Bütün giriş ve çıkış terminallerinin mümkün değer birleşimleri nelerdir?**

$$\begin{aligned}\exists i_1, i_2, i_3, o_1, o_2 \ (&Signal(In(1, C1)) = i_1 \\ &\wedge Signal(In(2, C1)) = i_2 \\ &\wedge Signal(In(3, C1)) = i_3 \\ &\wedge Signal(Out(1, C1)) = o_1 \\ &\wedge Signal(Out(2, C1)) = o_2).\end{aligned}$$

Bu sorgunun çözümleri devrenin **tam giriş–çıkış tablosunu** verir; yukarıda elle hesapladığımız sekiz satır elde edilir. Böylece devrenin bir toplama devresi olup olmadığını kontrol ederiz. Bu tür denetime **circuit verification (devre doğrulama)** denir. Devre tanımlarını yeniden kullanarak daha büyük dijital sistemler tasarlanabilir ve aynı çıkarım yaklaşımıyla doğrulanabilir.

Adım 7 — Bilgi tabanını hata ayıklama

Aksiyomları bilinçli olarak bozarsak hangi hataların ortaya çıktığını görebiliriz. 8.2.8'deki benzersiz isim tartışmasını unuttuğumuzu ve $1 \neq 0$ olgusunu eklemediğimizi varsayalım. Kaynak örnekte çıkarım sistemi `000` ve `110` girdileri dışındaki kombinasyonlar için belirli çıktıları kanıtlayamaz. Hangi kapıda sorun olduğunu bulmak için **X1'in çıktısı** tek başına sorgulanabilir. Girişler `10` veya `01` olduğunda çıktı ispatlanamıyorsa XOR aksiyomuna bakılır:

$$Signal(Out(1, X1)) = 1 \Leftrightarrow Signal(In(1, X1)) \neq Signal(In(2, X1)).$$

Girişler 1 ve 0 ise:

$$Signal(Out(1, X1)) = 1 \Leftrightarrow 1 \neq 0.$$

Standart FOL anlambiliminde **farklı semboller aynı nesneyi gösterebileceğinden**, bilgi tabanında $1 \neq 0$ yoksa sağ taraf ispatlanamayabilir. Dolayısıyla çıktı 1 de ispatlanamaz. Çözüm basittir: Aksiyomları açıkça tamamlayıp $1 \neq 0$ bilgisini eklemek.

> **Temel ders.** Bir olgunun insan açısından "zaten açık" olması, onun mantıksal bilgi tabanında kanıtlanabildiği anlamına gelmez. Güvenilir çıkarım, **açık ve yeterli aksiyomlar** gerektirir. Hata bazen kapı algoritmasında değil, alanın eksik tanımlanmış ontolojisindedir.

Ek A. Bütün Bölümü Birleştiren Çözümlü Örnekler

Bu ek, kaynak metindeki açıklamalara ek olarak öğrencilerin FOL gösterimleriyle çalışmasını kolaylaştırmak için hazırlanmıştır.

Örnek 1 — Genel cümleyi mantığa çevirme

“Bütün öğretmenler insandır.”

$$\boxed{\forall x\ (Teacher(x) \Rightarrow Person(x)).}$$

Burada evren bütün nesneleri içerebilir. İnsan olmayan nesneler veya öğretmen olmayan kişiler koşullu ifadenin öncülünü sağlamadığından kurala aykırı değildir.

Örnek 2 — En az bir nesne

“En az bir öğrenci matematik sever.”

$$\boxed{\exists x\ (Student(x) \land Likes(x, Math)).}$$

Var olan nesnenin **hem öğrenci olması hem matematiği sevmesi** gerekir; bu nedenle $\land$ kullandık.

Örnek 3 — Niceleyici sırası

“Her öğrencinin en az bir öğretmeni vardır”:

$$\forall x\ (Student(x) \Rightarrow \exists y\ (Teacher(y) \land Teaches(y, x))).$$

“Bütün öğrencileri eğiten tek bir ortak öğretmen vardır”:

$$\exists y\ (Teacher(y) \land \forall x\ (Student(x) \Rightarrow Teaches(y, x))).$$

İlkinde y, her öğrenciye göre farklı olabilir; ikincisinde y aynı kişidir.

Örnek 4 — Tam olarak iki nesne

“Ali’nin tam olarak iki arkadaşı vardır.”

$$\exists x, y\ (x \neq y \land Friend(x, Ali) \land Friend(y, Ali) \\ \land \forall z\ (Friend(z, Ali) \Rightarrow (z = x \lor z = y))).$$

Yalnızca iki arkadaşın varlığını belirtmek yetmez; başka arkadaş olmadığını da belirtmeliyiz.

Örnek 5 — Aksiyom mu teorem mi?

Aksiyom: “Ebeveyn ilişkisi çocuk ilişkisinin tersidir”:

$$\forall p, c\ (Parent(p, c) \Leftrightarrow Child(c, p)).$$

Özel olgu: $Parent(Ayse, Deniz)$. Teorem: $Child(Deniz, Ayse)$. Sonuç yeni bir aksiyom gerektirmeden ilk iki bilgiden çıkar.

Örnek 6 — Esinti ve belirsizlik

Kare [2,2]’de esinti vardır. Buna göre en az bir komşuda çukur vardır. Ancak **hangi komşuda** olduğunu ayrıca kanıtlayamıyorsak hiçbir komşuyu tek başına kesin çukur ilan etmeyiz. Bu, bilgi tabanının gerçeğe uygun biçimde **kısmi bilgiyi** sakladığını gösterir.

Örnek 7 — Tam toplayıcıda üç adet 1

$A = B = C_{in} = 1$ için:

$$X1 = 1 \oplus 1 = 0,$$
$$A1 = 1 \land 1 = 1,$$
$$X2 = 0 \oplus 1 = 1,$$
$$A2 = 0 \land 1 = 0,$$
$$O1 = 1 \lor 0 = 1.$$

Sonuç $S = 1, C_{out} = 1$ yani ikili $(11)_2 = 3$; toplama gerçekten $1 + 1 + 1 = 3$ verir.

Ek B. Hızlı Başvuru Sözlüğü

İngilizce	Türkçe	Not
First-Order Logic (FOL)	Birinci Dereceden Mantık	Nesne ve ilişki odaklı mantık
Propositional Logic	Önermeler Mantığı	Doğru / yanlış önermeleri kullanır
Knowledge Base (KB)	Bilgi Tabanı	Olgular ve kurallar
Domain	Evren / Problem Alanı	Modelin nesneleri veya ele alınan konu
Model	Model	Olası dünyanın biçimsel gösterimi
Interpretation	Yorumlama	Sembollerin karşılıkları
Constant Symbol	Sabit sembolü	Nesneyi adlandırır
Predicate	Yüklem	Özellik veya ilişki
Function	Fonksiyon	Girdiye tek değer eşler
Arity	Argüman sayısı	Yüklemin/fonksiyonun kaç argümanı var?
Term	Terim	Bir nesneye gönderme yapar
Atomic Sentence	Atomik cümle	Temel doğru/yanlış ifadesi
Quantifier	Niceleyici	$\forall, \exists$
Universal	Evrensel	Bütün nesneler için
Existential	Varoluşsal	En az bir nesne için
Substitution	Yerine koyma	$\{x/John\}$ gibi
Axiom	Aksiyom	Temel bilgi veya kural
Theorem	Teorem	Aksiyomlardan çıkan sonuç
Ontology	Ontoloji	Kavram ve ilişki sözlüğü
Knowledge Engineering	Bilgi Mühendisliği	Bilgi tabanı geliştirme süreci
Unique-Names Assumption	Benzersiz İsimler Varsayımı	Ayrı adlar farklı nesnelerdir
Closed-World Assumption	Kapalı Dünya Varsayımı	Bilinmeyen atomik olgular yanlış sayılır
Domain Closure	Evren Kapanışı	İsimsiz başka nesne yoktur
Ground Term	Değişkensiz terim	$LeftLeg(John)$ gibi
Successor-State Axiom	Ardıl Durum Aksiyomu	Durumun zamanla güncellenmesi
Circuit Verification	Devre Doğrulama	Beklenen giriş/çıkış davranışının denetimi

Ek C. Öğrenciler İçin Alıştırmalar ve Kısa Cevap Anahtarı

1. “Hiçbir öğrenci geç kalmamıştır” ifadesini FOL ile yazınız.
2. “Sınıfta en az iki farklı öğretmen vardır” ifadesini yazınız.
3. $\forall x \exists y\ Loves(x, y)$ ile $\exists y \forall x\ Loves(x, y)$ arasındaki farkı açıklayınız.
4. $\neg \forall x\ P(x)$ ifadesini $\exists$ kullanarak yazınız.
5. “Richard’ın iki kardeşi vardır” ifadesinde $x \neq y$ neden gereklidir?
6. $A = \{1, 2, 3\}$ ve $B = \{2, 3, 4\}$ için kesişimi bulunuz.
7. [A,B,A] listesi ile $\{A, B, A\}$ kümesi eşit sayıda öğe içerir mi?
8. Bir Wumpus karesinde esinti yoksa ilgili genel kural hangi komşuluk bilgisini sağlar?
9. Şekil 8.6’da $A = 0, B = 1, C_{in} = 1$ için S ve C_{out} değerlerini bulunuz.
10. Bir FOL modelinde `A` ve `B` farklı sabit sembolleri hangi koşulda aynı nesneyi gösterebilir?

Kısa cevap anahtarı:

1. $\forall x\ (Student(x) \Rightarrow \neg Late(x))$.
2. $\exists x, y\ (Teacher(x) \land Teacher(y) \land InClass(x) \land InClass(y) \land x \neq y)$.
3. İlki herkese farklı bir sevilen kişi verebilir; ikincisinde herkes için aynı sevilen kişi bulunur.
4. $\exists x\ \neg P(x)$.
5. Aksi halde x ve y aynı kişiyi gösterebilir; “iki” yerine “bir” kişiyle ifade doğrulanabilir.
6. $A \cap B = \{2, 3\}$.
7. Hayır. Listede üç konum vardır ve tekrar korunur; kümede iki farklı eleman vardır.
8. Komşu karelerin hiçbirinde çukur bulunmadığı sonucu çıkar (esinti–çukur çift yönlü kuralı altında).
9. $X1 = 1, A1 = 0, X2 = 0, A2 = 1, O1 = 1$; yani $S = 0$, $C_{out} = 1$.
10. Standart FOL’de eşitlikleri dışlayan aksiyom yoksa farklı sabitler aynı nesneyi gösterebilir; veritabanı anlambilimindeki benzersiz isim varsayımı buna izin vermez.

Kaynak ve kapsam notu: Bu ders notu, kullanıcı tarafından sağlanan *Artificial Intelligence: A Modern Approach*, 3. baskı, Bölüm 8’in **8.1, 8.2, 8.3 ve 8.4.2 dahil 8.4** alt bölümlerinin Türkçe aktarımı üzerine hazırlanmıştır. Verilen dört özgün şekil (8.2, 8.4, 8.5, 8.6) aynen kullanılmış; Şekil 8.1 ve 8.3 metin olarak yeniden düzenlenmiştir. Kutulu örnekler, tablolar, ilave çizimler, çözümlü sorular, açıklamalar ve sözlük **öğretimi kolaylaştırmak için eklenmiştir**. Bu dosya, kaynakta gönderilmeyen sonraki alt bölümlerin metnini içerdiğini iddia etmez.